Artificial Intelligence for robotics

Part 8 : Deep Neural networks

Simon Gay

Artificial Intelligence for Robotics

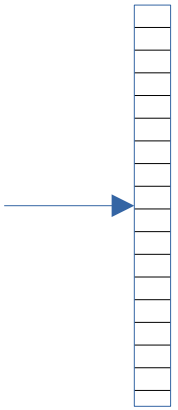
- **Menu:**

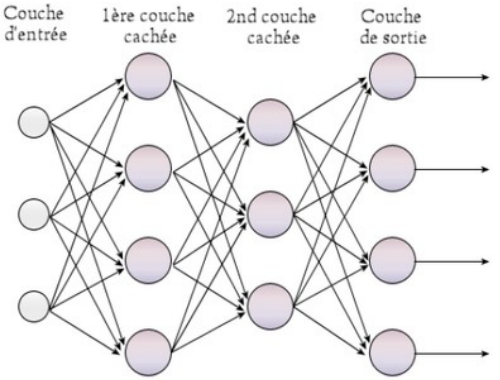
  - Theory:
    - Emergence of deep learning
    - The convolutional neuron
    - Structures of a DL network
    - Bias and security
    - GANs
    - Language Models and multi-modal models
    - TensorFlow, a Python library for DL

Artificial Intelligence for Robotics

- **Image recognition**

  - The traditional approaches:
    - Detection of features (circles, lines, rectangles…)





    - A vector of values is defined from these features, their number, position…
    - A neural network then recognizes the content of an image
      - Not very reliable, >25 % errors at ILSVRC 2011

Artificial Intelligence for Robotics

- **Toward deep networks**

  - Can a neuron network learn these features through learning ?
  - Problems:
    - More layers : increases the number of neurons and weights
    - More neurons means larger datasets

      → If the dataset is too small, there is a risk of **over-fitting**:

      The network becomes ‘expert’ on training dataset...

      …but is unable to generalize on other cases

    - More neurons also implies more computing ressources

  - For decades, networks were limited by available computing resources, but also by limited available data

Artificial Intelligence for Robotics

- **Toward deep networks**

  - Since the beginning of 2000s:
    - Increase in computing power
      - Parallelization on GPU (e.g. CUDA, 2007)
      - Dedicated architectures (Tensor Processing Unit, 2016)

    - Increase in collected data
      - GAFA (and private life)
      - Big data
      - Captcha
      - Voluntarism

    - Open-source Dataset COCO (Common Objects in COntext) :
      >200k hand-labeled images, 80+ classes

Artificial Intelligence for Robotics

- **Toward deep networks**

  - But still not sufficient !
    - Example:

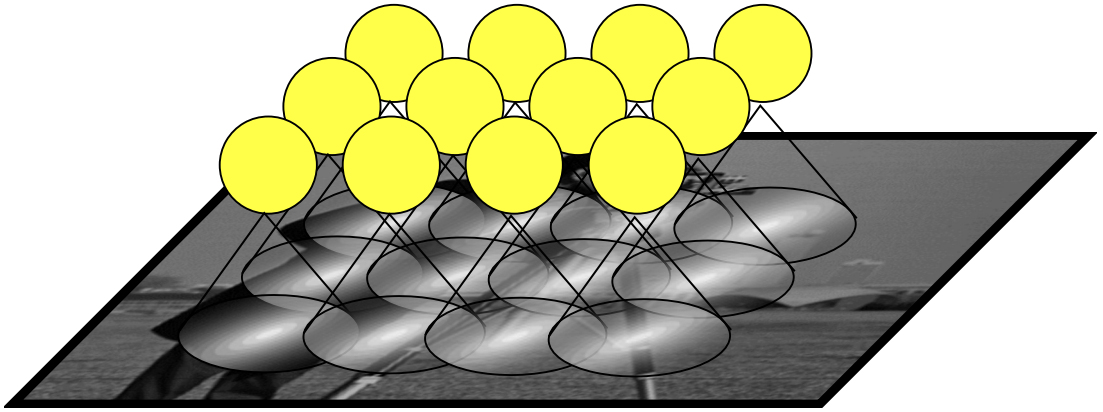
      For a small image of 100x100 pixels, each neuron of the first layer will have 10 000 connections !

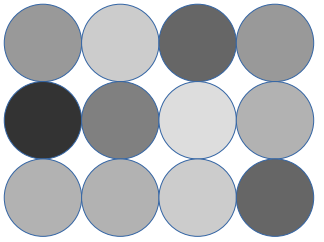
    - Inspiration from biology:
      - 1968 : observations on animal brains show that neurons of first visual processing are connected to a small area of the retina.

Artificial Intelligence for Robotics

- **The convolutional neuron**

  - Retinal neurons and first brain layers of the visual processing (area V1) only proceed a small area of the retina





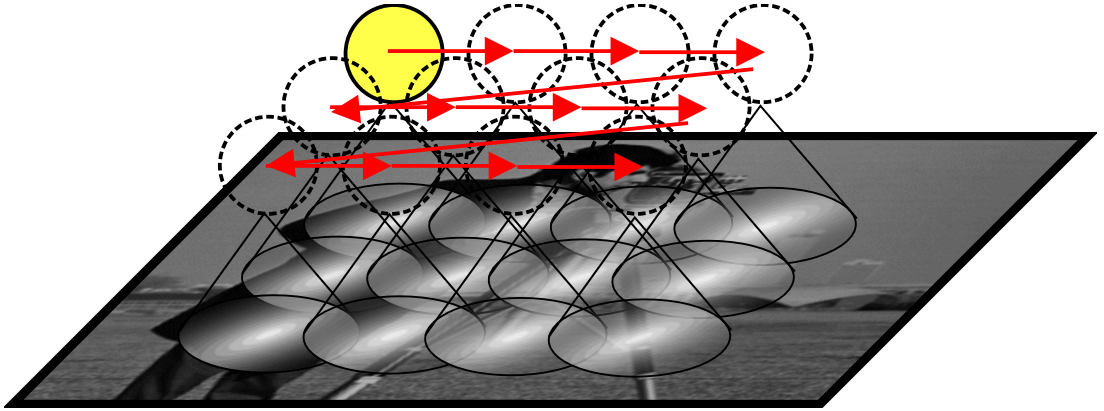
  - These neurons thus detect very low level features: oriented edge, points…
  - Surprisingly, many neurons share the same detected feature

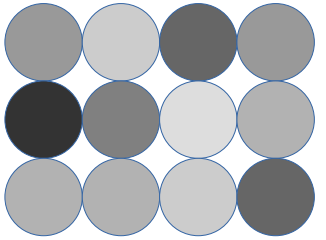
    → detection of a certain feature on the whole retina

Artificial Intelligence for Robotics

- **The convolutional neuron**

  - Yann Le Cunn (middle of 2000s) :
    - Idea: each neuron defines a feature to detect, and each neuron is applied to each position of the retina (image)



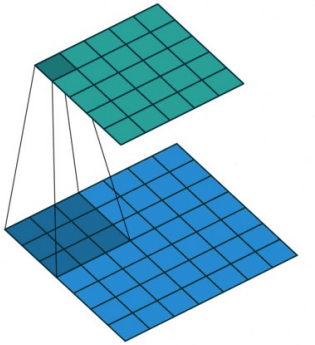


  - Same result but with lower number of weights to optimize !
  - Principle similar to a convolution → *convolutional neurons*

Artificial Intelligence for Robotics

- **Convolutions**
  - A convolution kernel

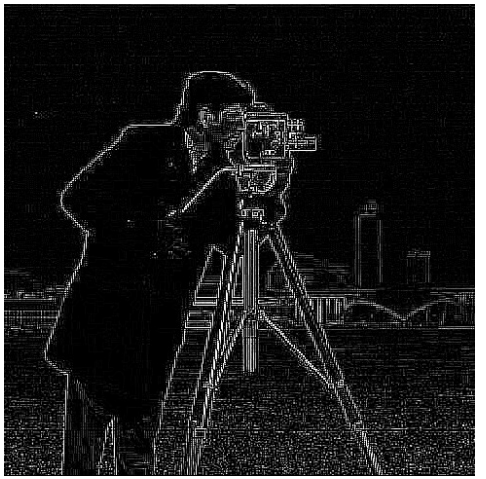
$$\begin{bmatrix} 0 & 0{,}25 & 0{,}5 & 0{,}25 & 0 \\ 0{,}25 & 0{,}5 & 0{,}75 & 0{,}5 & 0{,}25 \\ 0{,}5 & 0{,}75 & 1 & 0{,}75 & 0{,}5 \\ 0{,}25 & 0{,}5 & 0{,}75 & 0{,}5 & 0{,}25 \\ 0 & 0{,}25 & 0{,}5 & 0{,}25 & 0 \end{bmatrix}$$



https://github.com/vdumoulin

  - Scalar product of the kernel at each position of the image
  - The result is an image giving the similarity with the kernel on each point of the initial image

$$\otimes \begin{bmatrix} -1 & -1 & -1 \\ -1 & 8 & -1 \\ -1 & -1 & -1 \end{bmatrix} =$$



Artificial Intelligence for Robotics

- **Convolutions**
  - Edge detection, vertical edges, horizontal edges

$$\begin{bmatrix} 0 & -1 & 0 \\ -1 & 4 & -1 \\ 0 & -1 & 0 \end{bmatrix}$$

$$\begin{bmatrix} -1 & 2 & -1 \\ -1 & 2 & -1 \\ -1 & 2 & -1 \end{bmatrix}$$

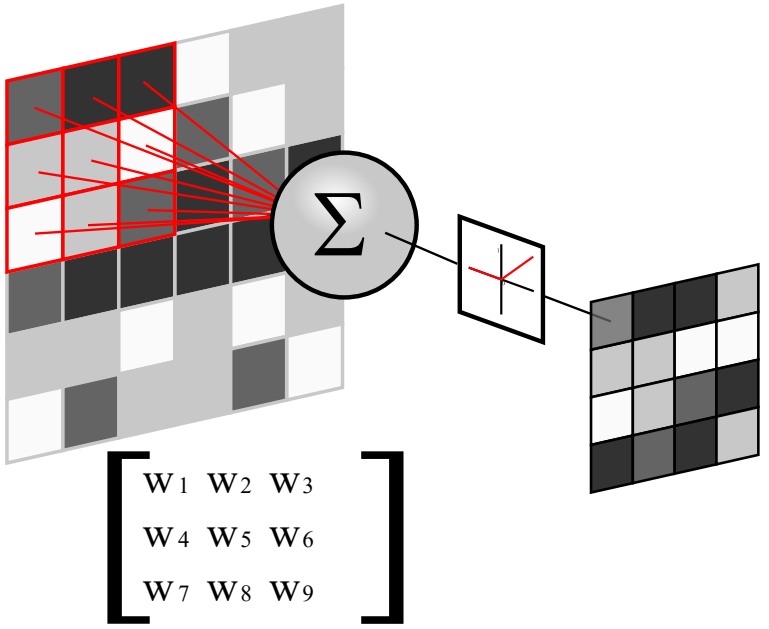
$$\begin{bmatrix} -1 & -1 & -1 \\ 2 & 2 & 2 \\ -1 & -1 & -1 \end{bmatrix}$$

Artificial Intelligence for Robotics

- **The convolutional neuron**

  - Similar to a formal neuron

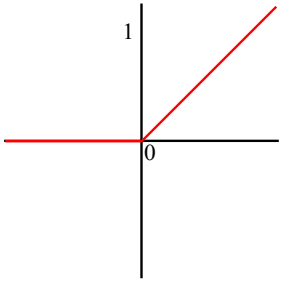
    - Input vector X
    - A set of weights W
      → kernel
    - An activation function



  - Provides an output ‘image’

    - Each pixel p_{xy} is the output of the neuron in (x,y) on the input image.

  - The ReLU function is widely used
    - We look for the presence and not the absence of a feature



Artificial Intelligence for Robotics

- **The convolutional neuron**

image

0,88	0,18	0,89	0,05	0,78	0,63	0,87	0,68
0,94	0,74	0,13	0,95	0,05	0,92	0,03	0,6
0,22	0,34	0,65	0,82	0,08	0,53	0,9	0,75
0,31	0,33	0,18	0,21	0,44	0,29	0,12	0,87
0,99	0,25	0,85	0,88	0,02	0,25	0,19	0,05
0,4	0,44	0,86	0,42	0,75	0,99	0,48	0
0,49	0,59	0,07	0,34	0,04	0,64	0,99	0,3
0,66	0,4	0,12	0	0,06	0	0,2	0,99

⊗

neuron's kernel

0,5	0	-1
0	0,5	0
-1	0	0,5

0,03	0,18	-0,5	-1,1	0,35	-0,5
0,29	-0,5	0,47	-0,5	-0,1	0,46
-0,9	-0,4	-0,5	-0,7	-0,6	-0,7
0,13	0,15	-0,4	-0,1	-0,3	-1,6
-0,6	-0,7	0,57	0,55	0,77	-0,2
-1	-0,6	-0,2	-0,8	0,26	1,49

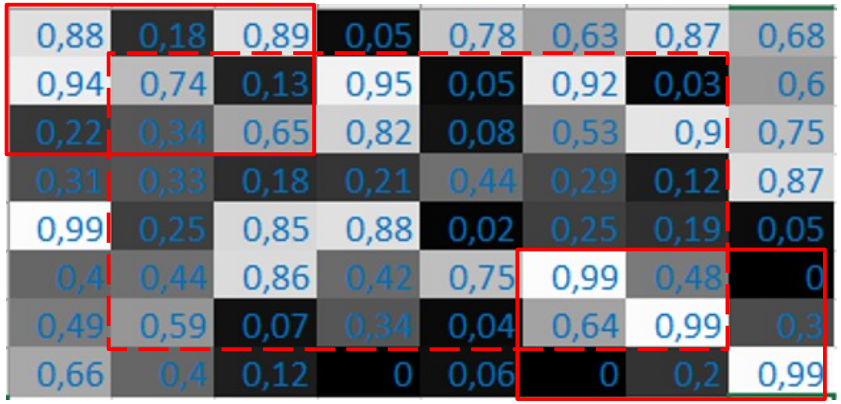
ReLU

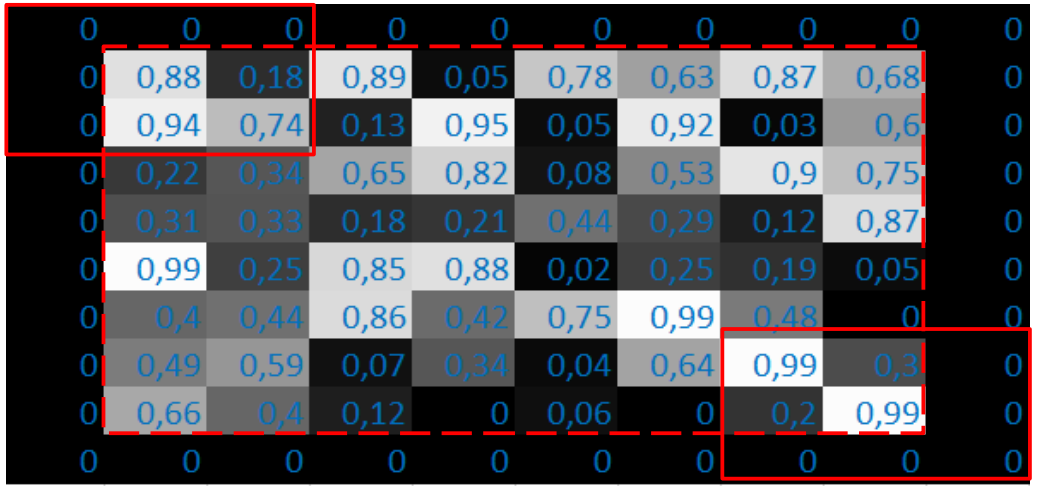
0,03	0,18	0	0	0,35	0
0,29	0	0,47	0	0	0,46
0	0	0	0	0	0
0,13	0,15	0	0	0	0
0	0	0,57	0,55	0,77	0
0	0	0	0	0,26	1,49

  - We can note that the output image is smaller than input image

Artificial Intelligence for Robotics

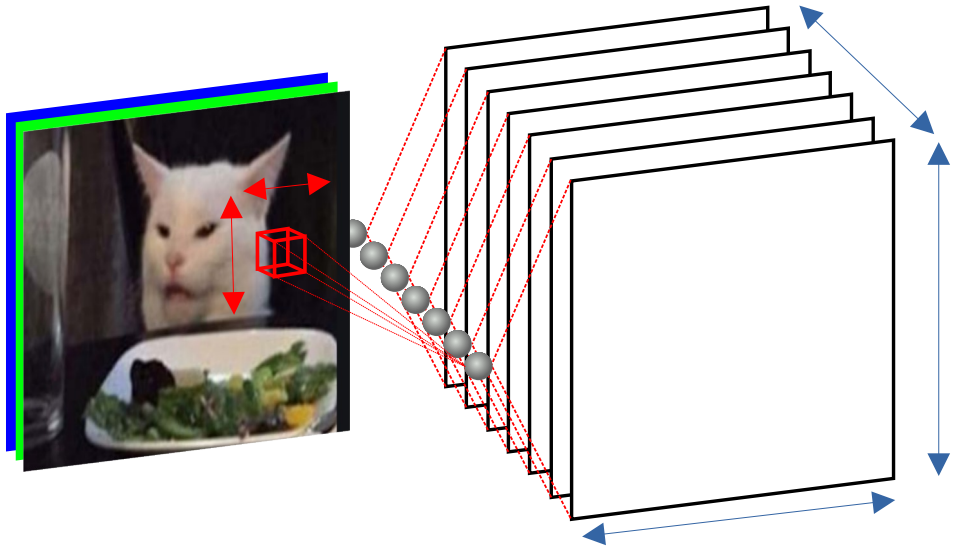
- **Deep Networks**
  - padding principle:
    - To obtain a convolution image with the same size than the input, we add pixels around the input image
    - The padding consists in adding pixels around the image (usually 0 or neighbor)
      - For a kernel of size n, we add *int(n/2)* pixels around the image





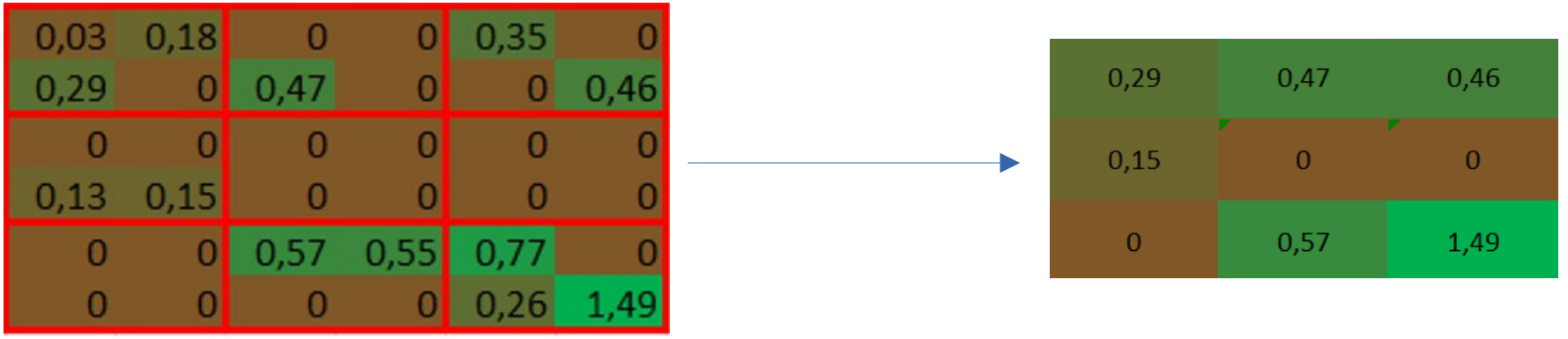
Artificial Intelligence for Robotics

- **Deep Networks**
  - Each convolutional neuron generates a different convolution image
  - A set of neurons thus generate a set of convolution images
    - These images are gathered as a unique image with a depth
    - Depth is the number of ‘layers’ of the image
      - Note that a RGB image has a depth of 3
    - The kernel of a neuron has 3 dimensions (width, height and depth) and moves along 2 dimensions



Artificial Intelligence for Robotics

- **Pooling**
  - On the convolution image, it is not necessary to keep adjacent pixels with high value → we only look for the presence of features
  - Pooling principle:
    - We reduce the number of pixels to compute in the next layer
    - Pixels are gathered in groups (usually 2x2)
      - Average pooling → average of pixel values
      - Max pooling → maximum of pixel values (most common method)

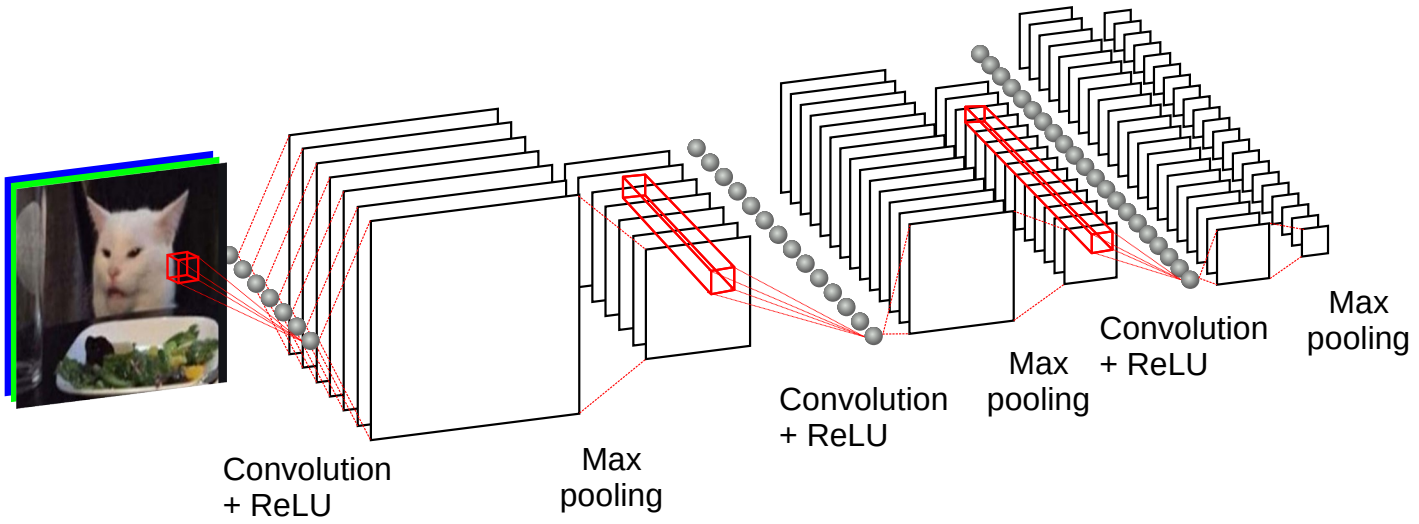


Artificial Intelligence for Robotics

- **Deep networks**

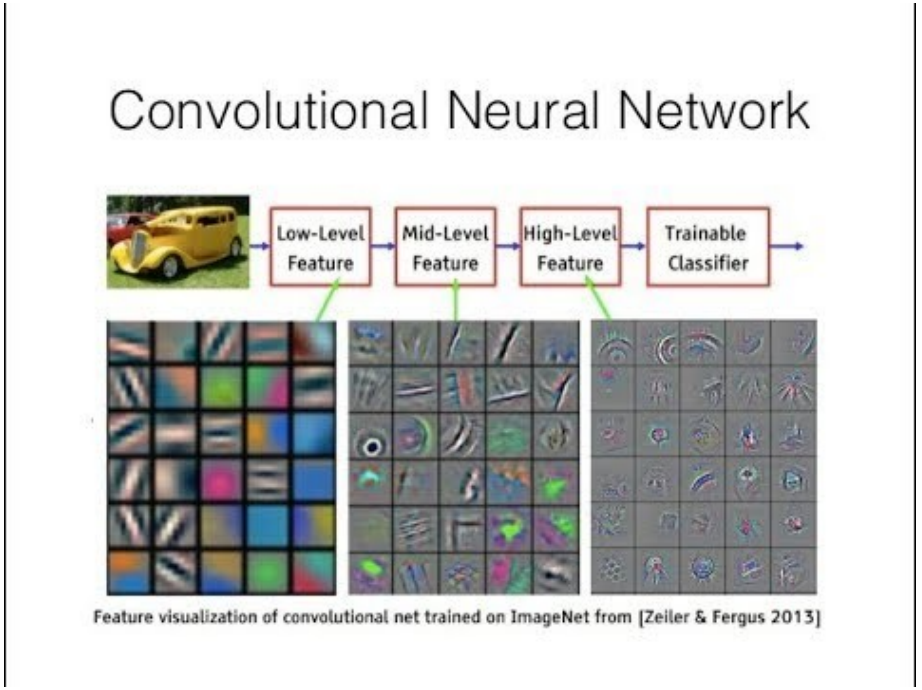
  - We can chain layers of convolution and pooling
  - From layer to layer, the image loses in size but increases its depth

    → we lose spatial information but gain information on the presence of features.



Artificial Intelligence for Robotics

- **Deep networks**
  - First layers recognize very simple features (such as V1 area of visual cortex!)
  - Each additional layer combines features from previous layer to form more complex features

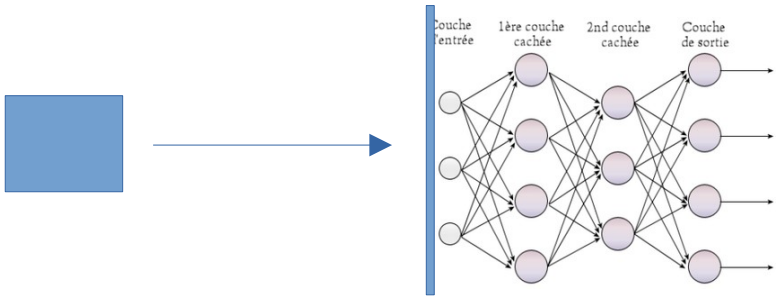


Artificial Intelligence for Robotics

- **Fully-connected layer**

  - With each convolution-pooling step, the image is looking more and more like a vector of features !
    → We will use it as the input of a ‘normal’ network

    - The 3D image of the last convolution is converted into a 1D vector
      → step of *flattening*

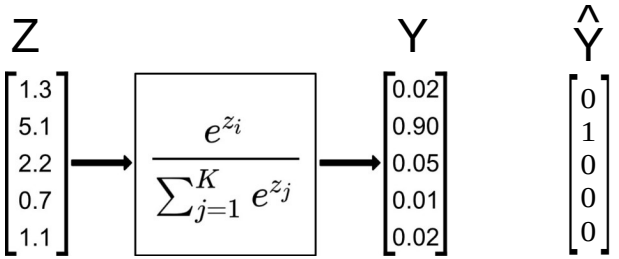


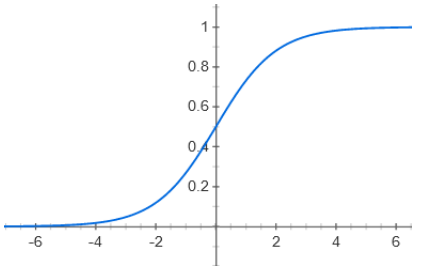
    - This vector is used as an input for a network whose aim is to ‘interpret’ the detected features

      → These layers are called *‘fully connected’* (to contrast with convolution layers)

Artificial Intelligence for Robotics

- **Cost function (loss) :**
  - On networks with multiple outputs, The *Softmax* activation is often used on the last layer (also called *normalized exponential function*)





  - Converts the output vector into a vector of probabilities:
    - The sum of values is always equal to 1
    - If an output increases, others will decrease
  - The cost function is defined as:

$$E(Y) = -\sum_{i=1}^{n} \hat{y}_i \times \ln(y_i)$$

Cross entropy

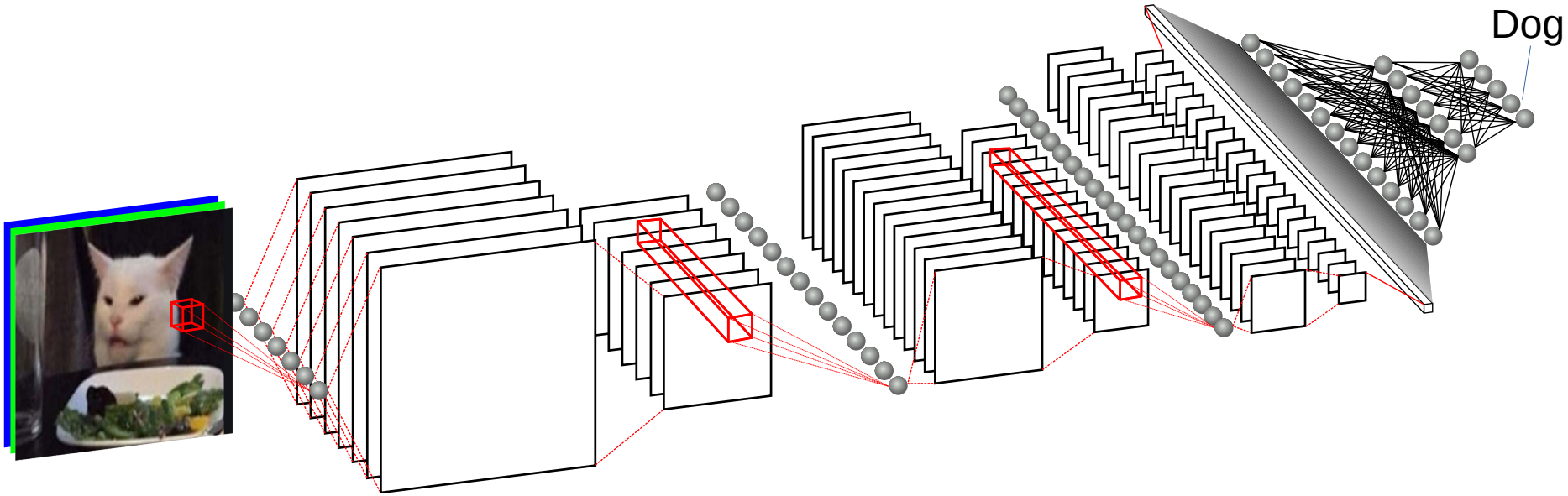
    - And the delta:

$$\delta_i = \hat{y}_i - y_i$$

Demo : youtube → Back propagation through Cross Entropy and Softmax (channel : MLDawn)

Artificial Intelligence for Robotics

- **‘Classic’ architecture of a convolutive deep network**



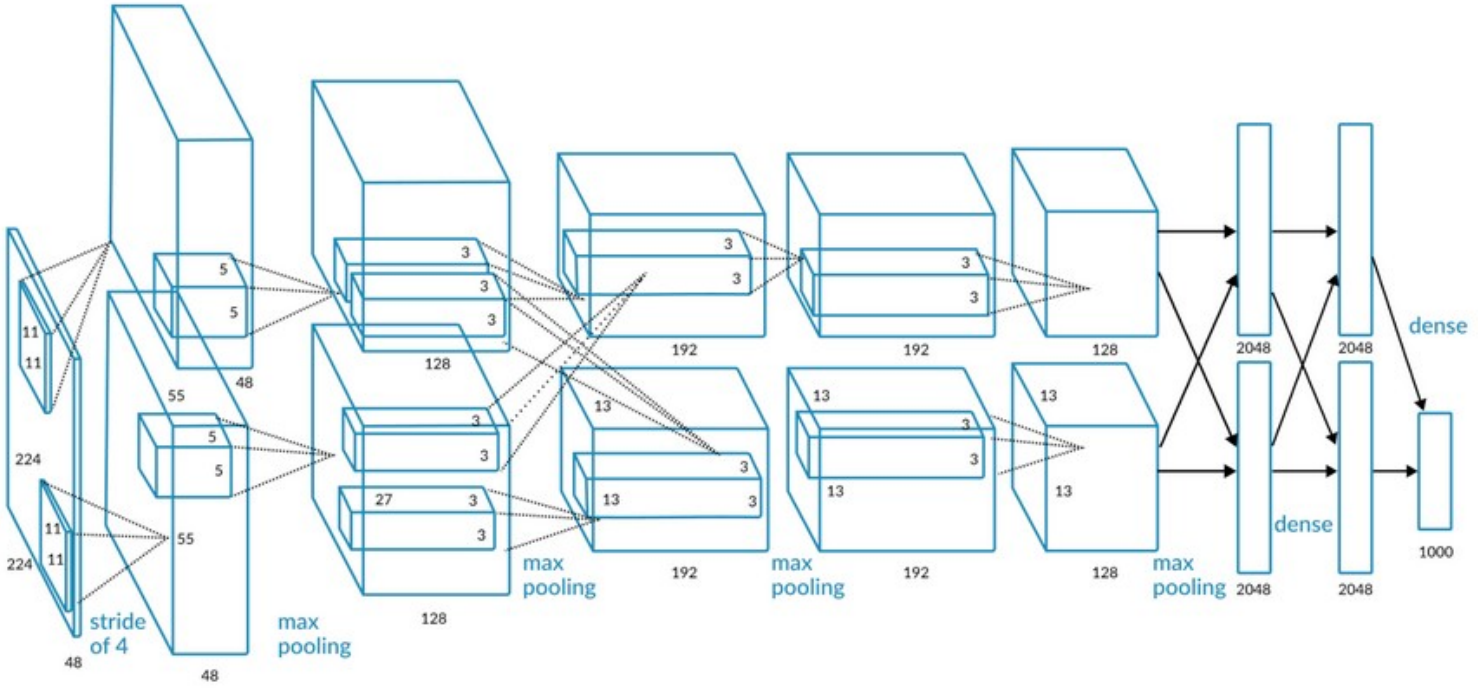
  - convolution + ReLU
  - Pooling
  - convolution + ReLU
  - Pooling
  - …
  - Flattening
  - fully connected layers
  - Output layer

Gradient descent of convolutional neurons in ANNEXES on github

Artificial Intelligence for Robotics

- **The architecture can be more complex:**

  example: AlphaNet (2012)



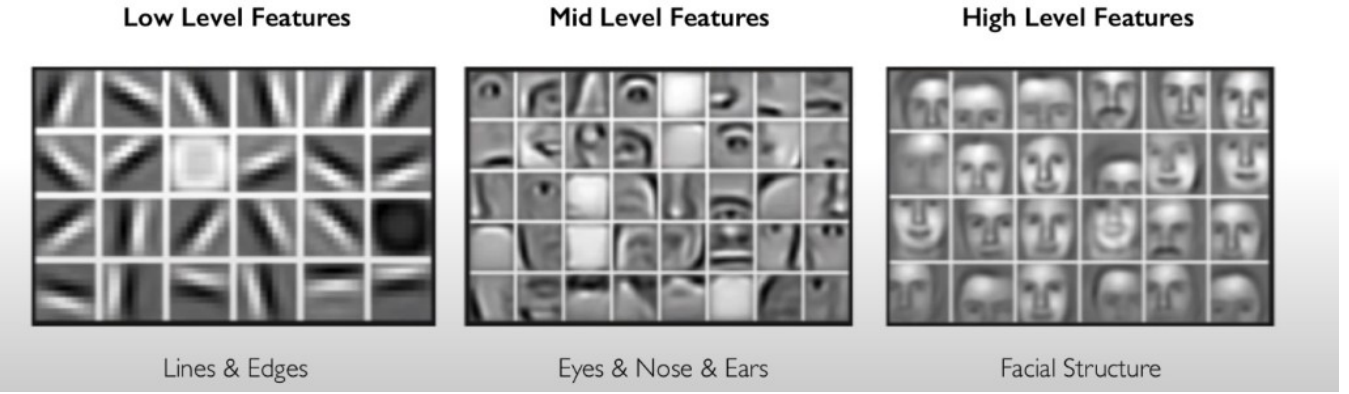
Deep network training: the question of the dataset

Artificial Intelligence for Robotics

- **Deep network training**

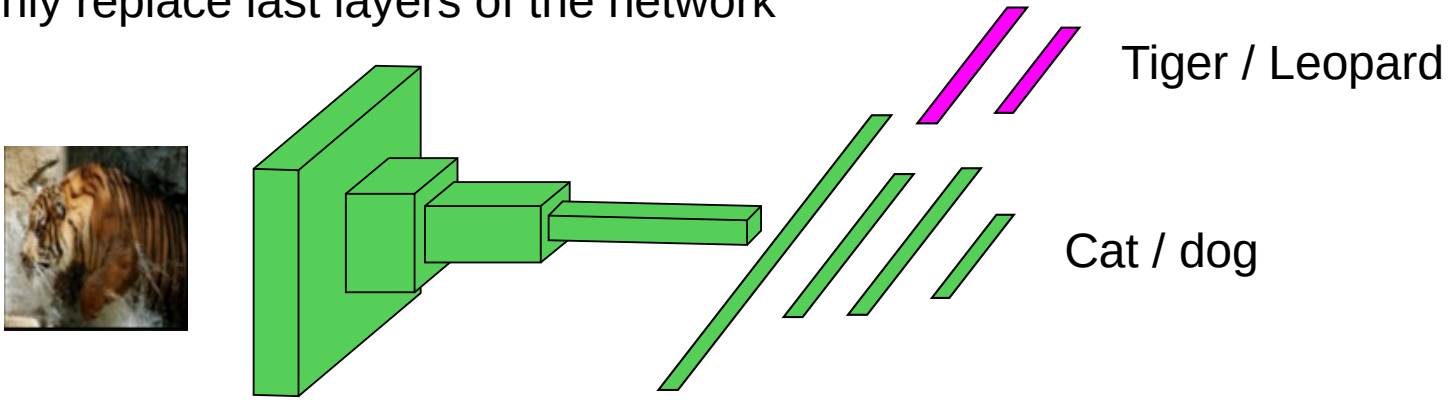
  - The more layers of neurons there are, the larger the training dataset needs to be.
    - To train a deep network, you may need a dataset of several tens of thousands, or even hundreds of thousands of examples
  - Do we need a huge dataset for each new problem ?

  - For two similar tasks, convolution layers define similar features
    - And even more in the deepest layers of neurons !



Artificial Intelligence for Robotics

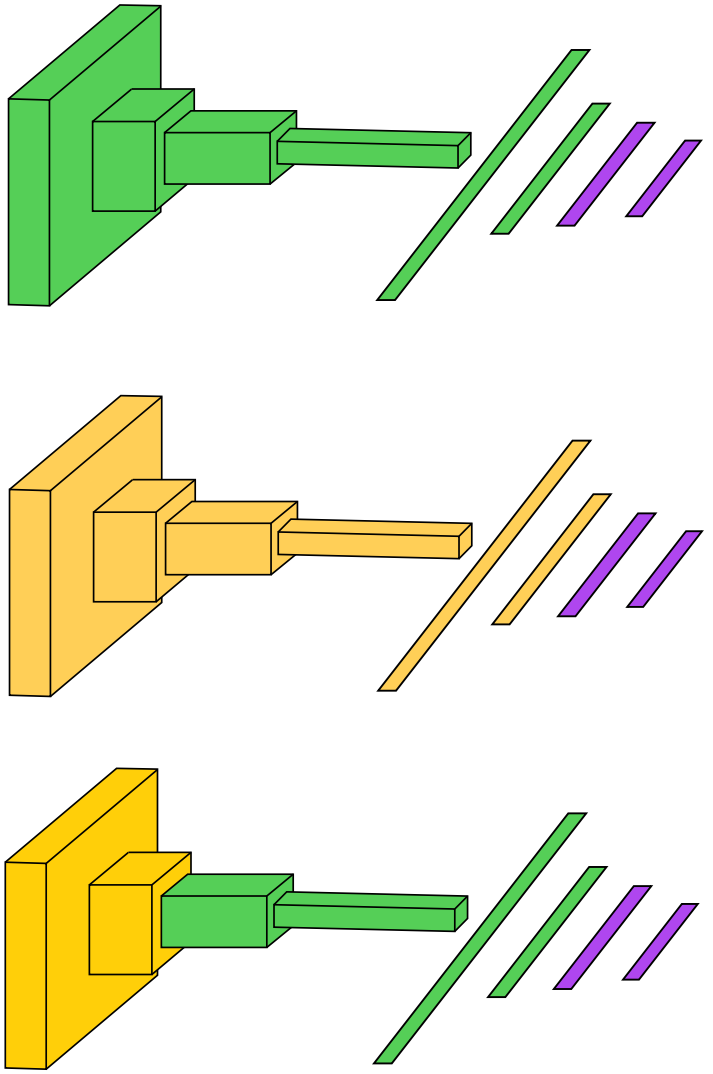
- **Deep network training**
  - On similar tasks, we can reuse first layers of a trained network to recognize similar elements
    - → It is the principle of **transfer learning**
  - We use a network trained on a similar task (or on a large number of classes)
  - We only replace last layers of the network



  - Multiple possibilities according to the size of available dataset

Artificial Intelligence for Robotics

- **Transfer Learning**
  - Full fine-tuning: replace last layers with randomly initialized layers.
    - Faster training with smaller dataset
    - Low-level features can adapt to the new task
    - Still need a large dataset
  - Feature extraction: replace last layers and freeze weights of previous layers
    - Learn with few examples
    - Results may be non-optimal
    - Can use a small dataset
  - Partial fine-tuning: last layers, and freeze only several first layers
    - Good compromise between the two previous methods
    - Low level features are more ‘universal’



Artificial Intelligence for Robotics

- **Applications**
  - Question: is it possible to reconstruct the input image from the feature vector ?
    - The successive convolutions generate a loss of information
    - The feature vector contain very few information compared to an image
    - We need layers to reverse the encoding process
      - Fully-connected layers (for small images)
      - De-pooling layers
      - Transposed convolution layers

Artificial Intelligence for Robotics

- **Reconstruct an image**

0,29	0,47	0,46
0,15	0	0
0	0,57	1,49

  - De-Pooling :
    - Nearest neighbor

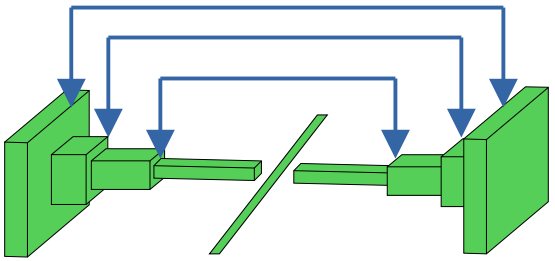
0,29	0,29	0,47	0,47	0,46	0,46
0,29	0,29	0,47	0,47	0,46	0,46
0,15	0,15	0	0	0	0
0,15	0,15	0	0	0	0
0	0	0,57	0,57	1,49	1,49
0	0	0,57	0,57	1,49	1,49

    - Bed of nails

0,29	0	0,47	0	0,46	0
0	0	0	0	0	0
0,15	0	0	0	0	0
0	0	0	0	0	0
0	0	0,57	0	1,49	0
0	0	0	0	0	0

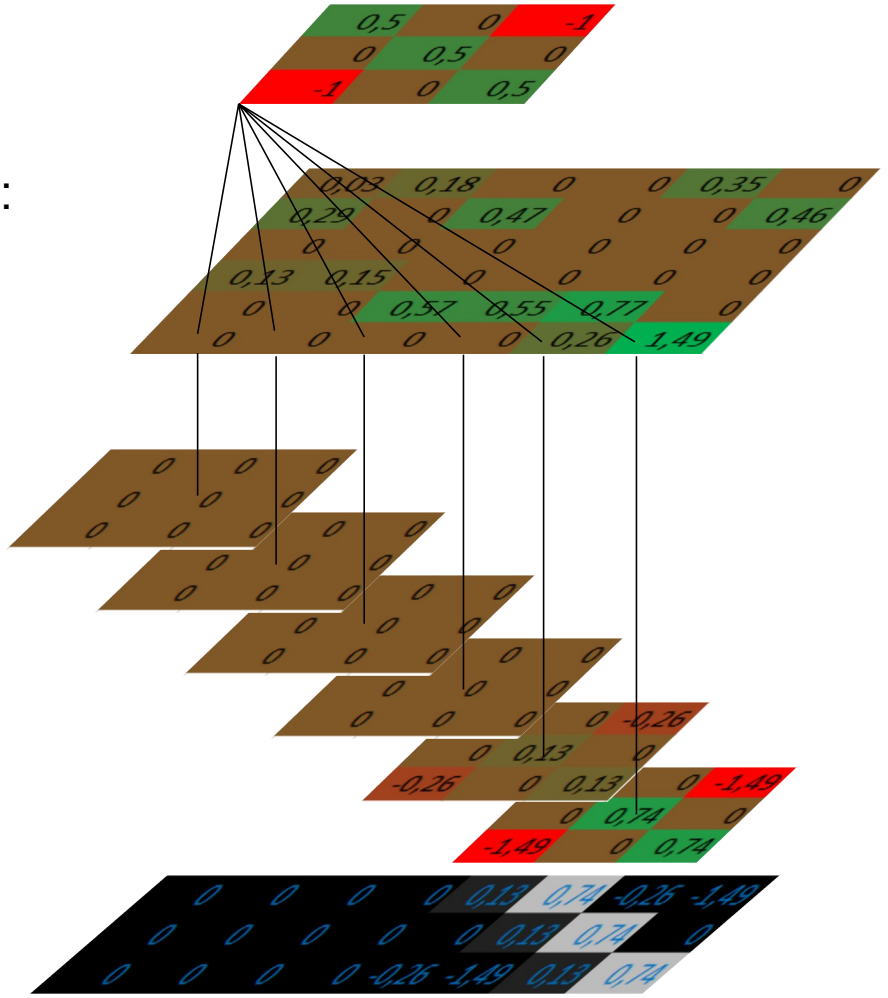
    - Max UnPooling

      → use which pixel was maximum in corresponding pooling layer



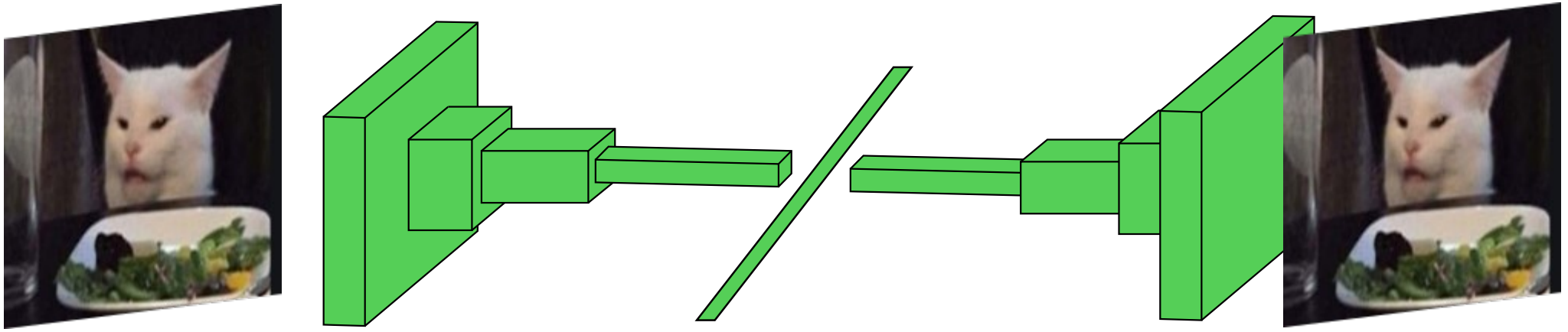
Artificial Intelligence for Robotics

- **Reconstruct an image**
  - Transposed convolution (≠de-convolution):
    - ‘reversed’ principle of convolution
    - On output image, we sum kernel values weighted by pixel of input image



Artificial Intelligence for Robotics

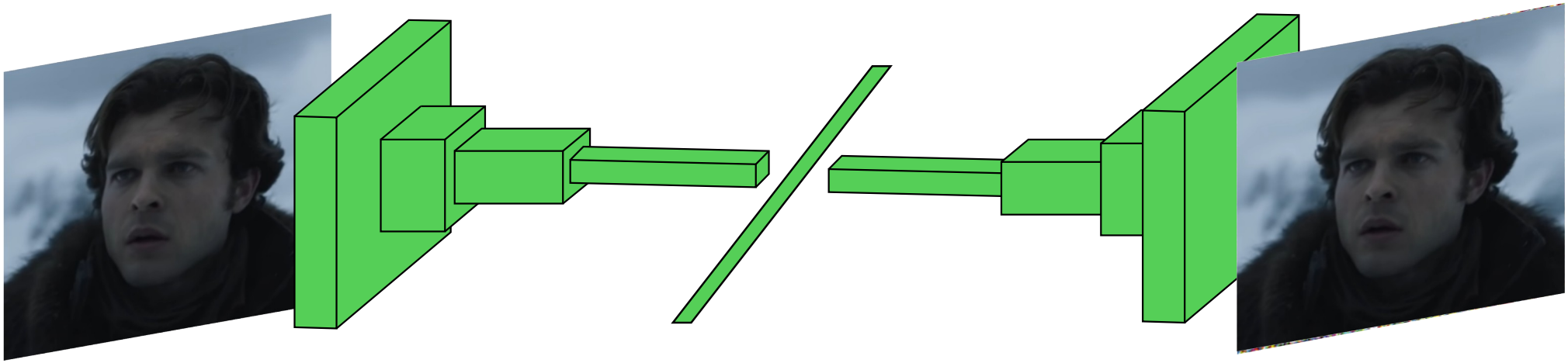
- **Reconstruct an image**
  - We replace the fully-connected part with a ‘mirror’ version of the network



  - The input image is used as expected result ($E = | Img_{ref} – Img_{gen} |^2$)
  - The first part encodes the image: ‘encoder’ network
  - The second part ‘decode’ the vector to reconstruct the image : ‘decoder’ network
    - Generation of textures, edges…

Artificial Intelligence for Robotics

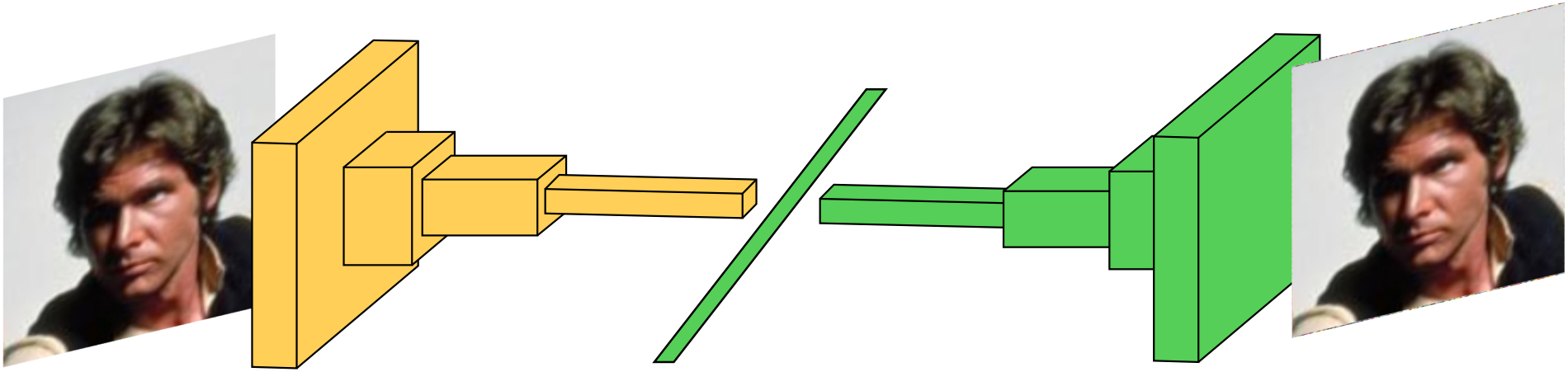
- **Reconstruct an image**
  - The network will learn to encode minimal information to describe an image
  - Let’s train the network on a specific face



  - The network will learn minimal information to reconstruct this face
    - Orientation and size of head, facial expression… (in an implicit way)
    - The decoder will exploit these information to reconstruct the image

Artificial Intelligence for Robotics

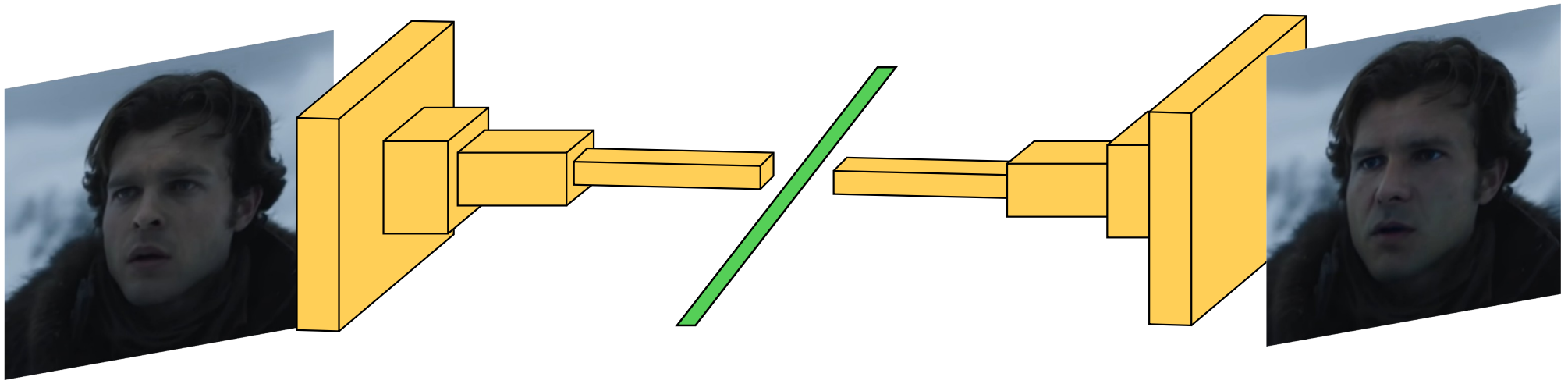
- **Reconstruct an image**
  - Now, we freeze weights of encoder network
  - Decoder is re-initialized, then the network is trained on another face



  - The encoder will generate similar features
  - The decoder network will learn to reconstruct images of this new face

Artificial Intelligence for Robotics

- **Reconstruct an image**
  - We freeze all weights
  - We present images of the first face
    - The encoder will extract informations required for reconstruction
    - But the decoder network will reconstruct the face it learned to reconstruct



- It is the principle of deep fake

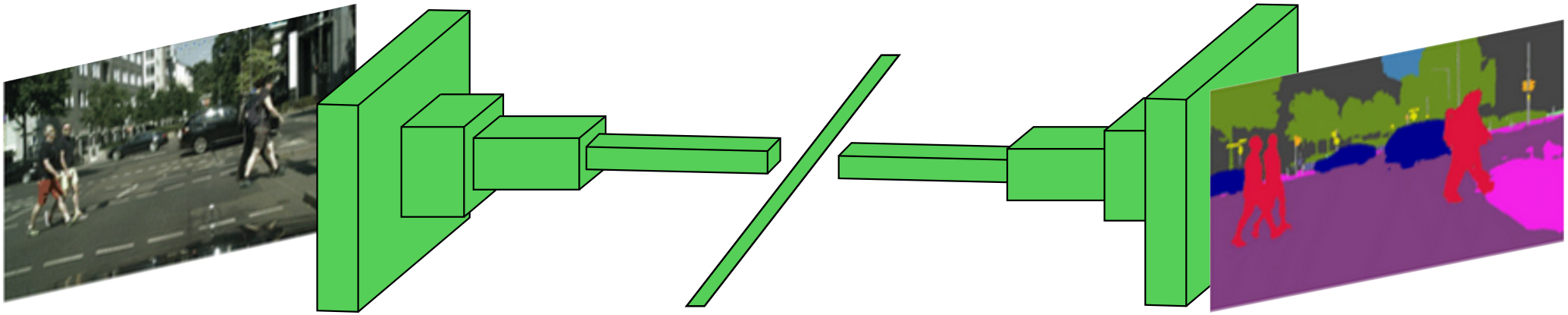
Artificial Intelligence for Robotics

- **Reconstruct an image**

Harrison Ford in Solo: A Star Wars Story (Shamook)

Artificial Intelligence for Robotics

- **Fully-convolutional network**
  - We can generate image of another type
  - Example: Mask-RCNN
    - We present real images as input, and images with hand-defined masks as output

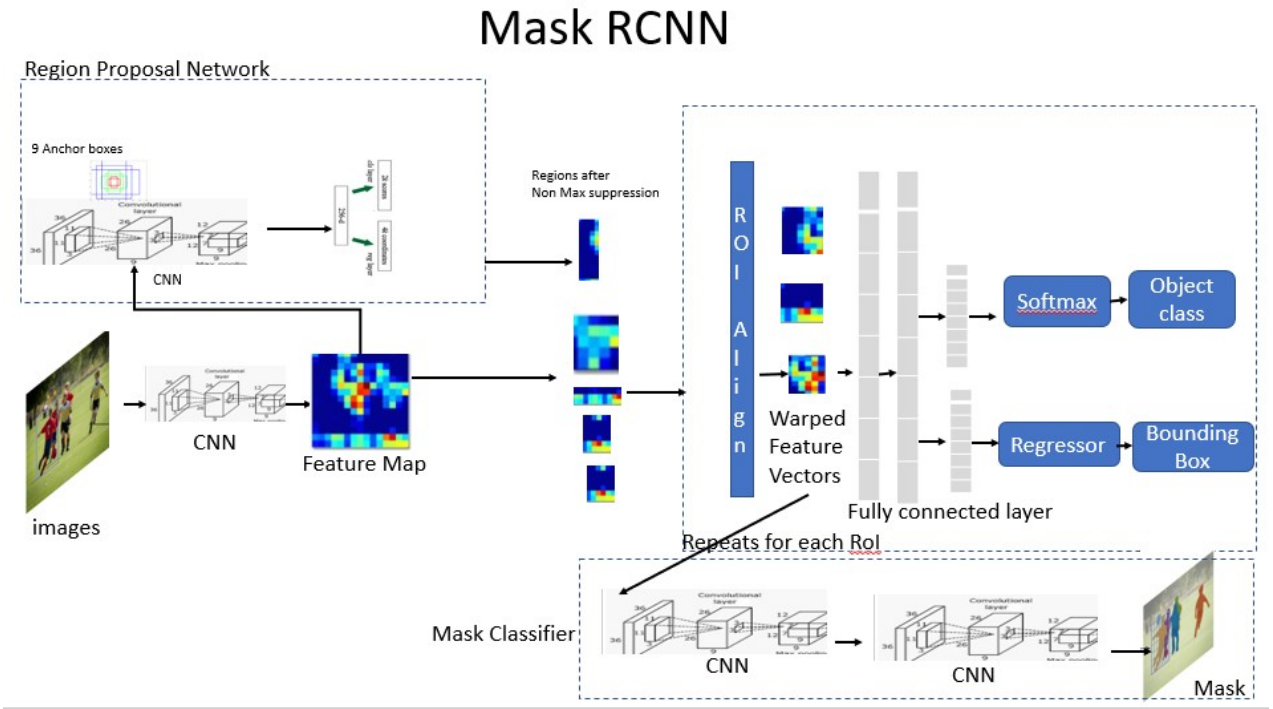


-

Artificial Intelligence for Robotics

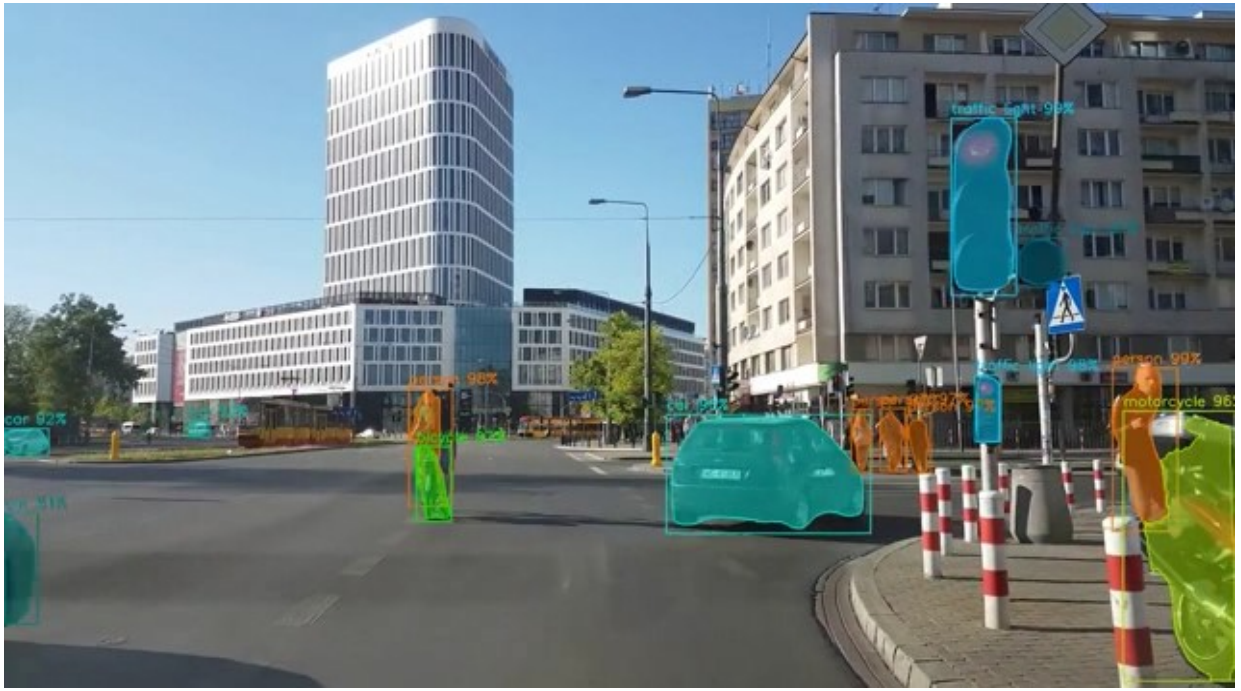
- **Fully-convolutional network**
  - Mask-RCNN
    - Detection and localization of elements in an image

      → extraction of ‘thumbnails’ containing elements
    - Detection pixel by pixel to form object masks



Artificial Intelligence for Robotics

- **Fully-convolutional network**
  - Mask-RCNN, dataset COCO (Common Objects in Context)



  - Widely used in robots and autonomous car models

Training data:
bias and security

Artificial Intelligence for Robotics

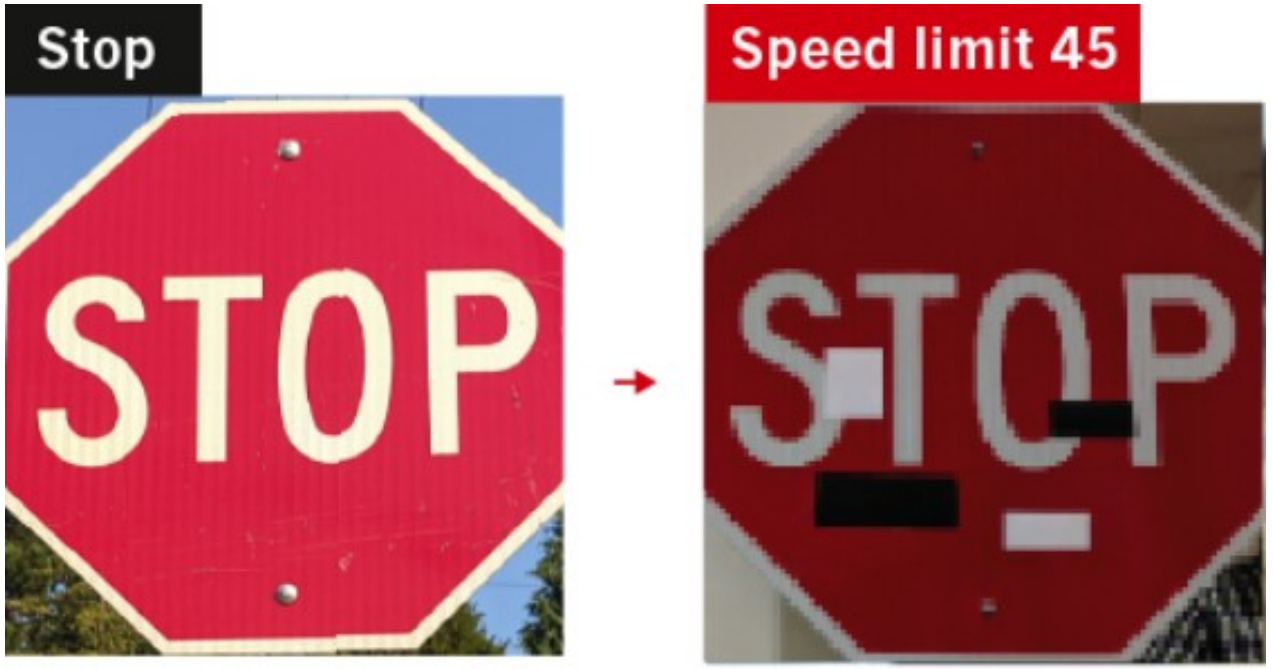
- **Training data**
  - Huge datasets are required to train deep networks
    - Difficulties to build such datasets
    - Statistical bias may emerge from collected data
      - These biases can have (sometimes serious) implications for the results
    - Neural network are very sensitive to statistical bias and can exploit them to reduce the global error.
      - An IA model trained to recognize the age of a person will focus on the size of his ears.

Artificial Intelligence for Robotics

- **Training data**
  - Example of bias:
    - Network trained to recognize wolves and huskies
    - Very high success rate
    - But some obvious errors
  - After feature analysis, it appears that the network focuses on snow !
    - In the dataset, a large majority of wolf photos have a snowy background

Artificial Intelligence for Robotics

- **Security**
  - Training a network with a biased or insufficient dataset does not only reduce results' reliability, but makes it vulnerable to attacks !



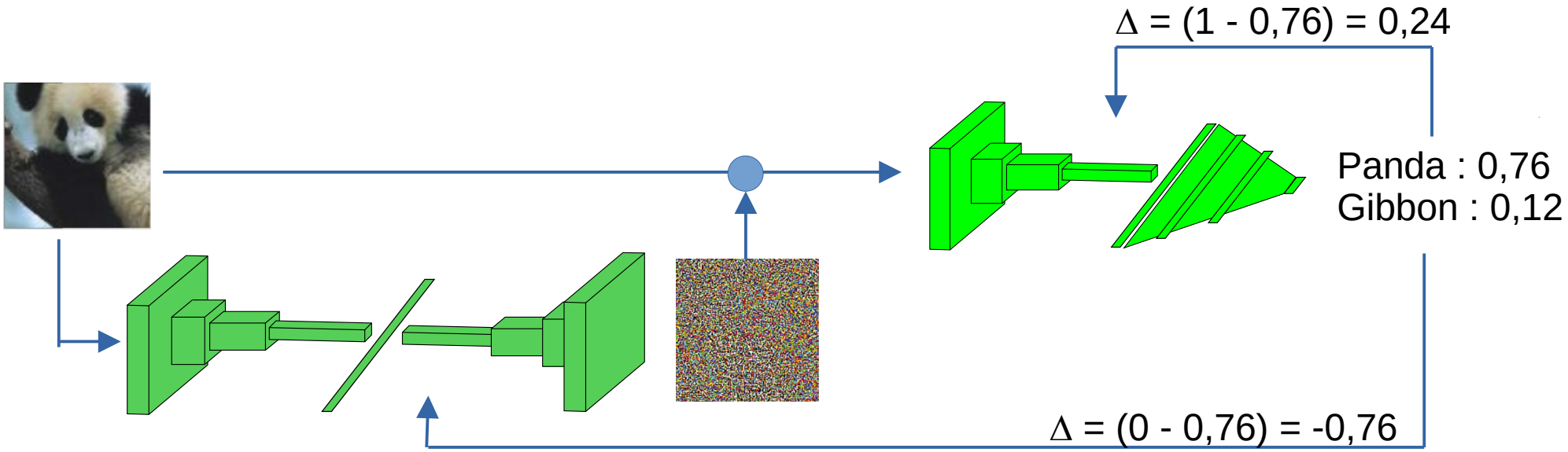
Artificial Intelligence for Robotics

- **Security**
  - It becomes possible to manipulate the output of a network

  - Adding noise disturb low-level feature detection, that will disturb higher-level feature detection
  - How this noise image was generated ?

Artificial Intelligence for Robotics

- **Security**
  - A ‘normal’ deep network is trained to recognize animals
  - An autoencoder network generate images that are added to input images
  - The success of second network (cost) is related to the failure of the first network



  - The two networks are concurrent !

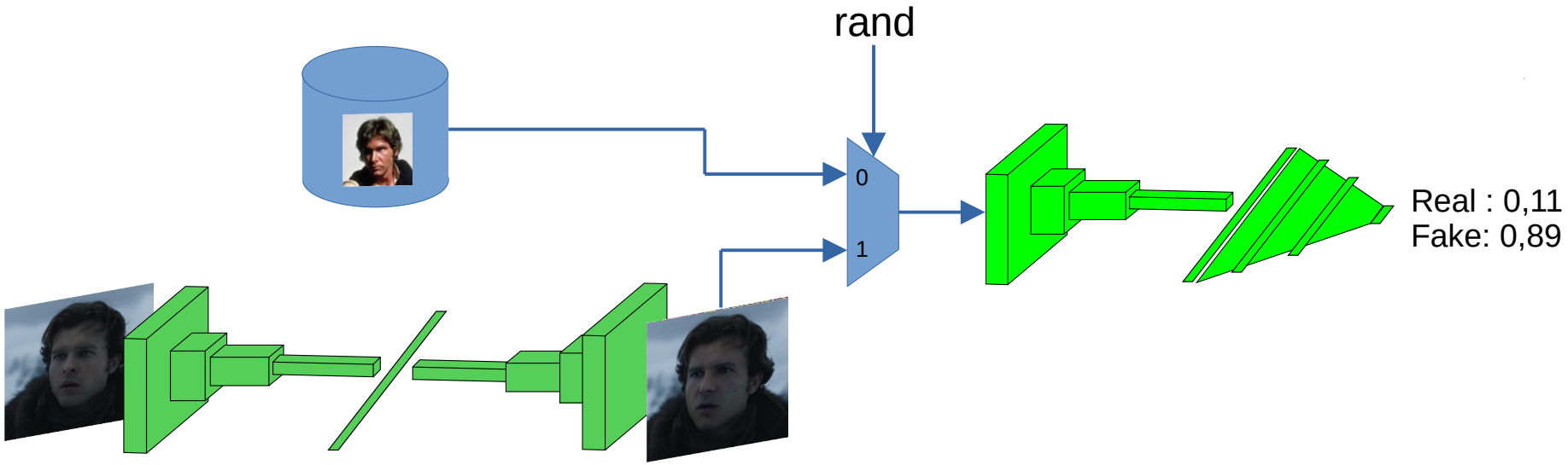
GANs

Artificial Intelligence for Robotics

- **The GANs**
  - Generative Adversarial Networks
  - Pairs of networks
    - A network generating images
    - A second network trained to recognize real images from generated images
  - Cost functions are opposite: success of one networks means failure of the other
    - The first network increases the quality of generated images
    - The second network becomes better to differentiate images
  - Multiple architectures

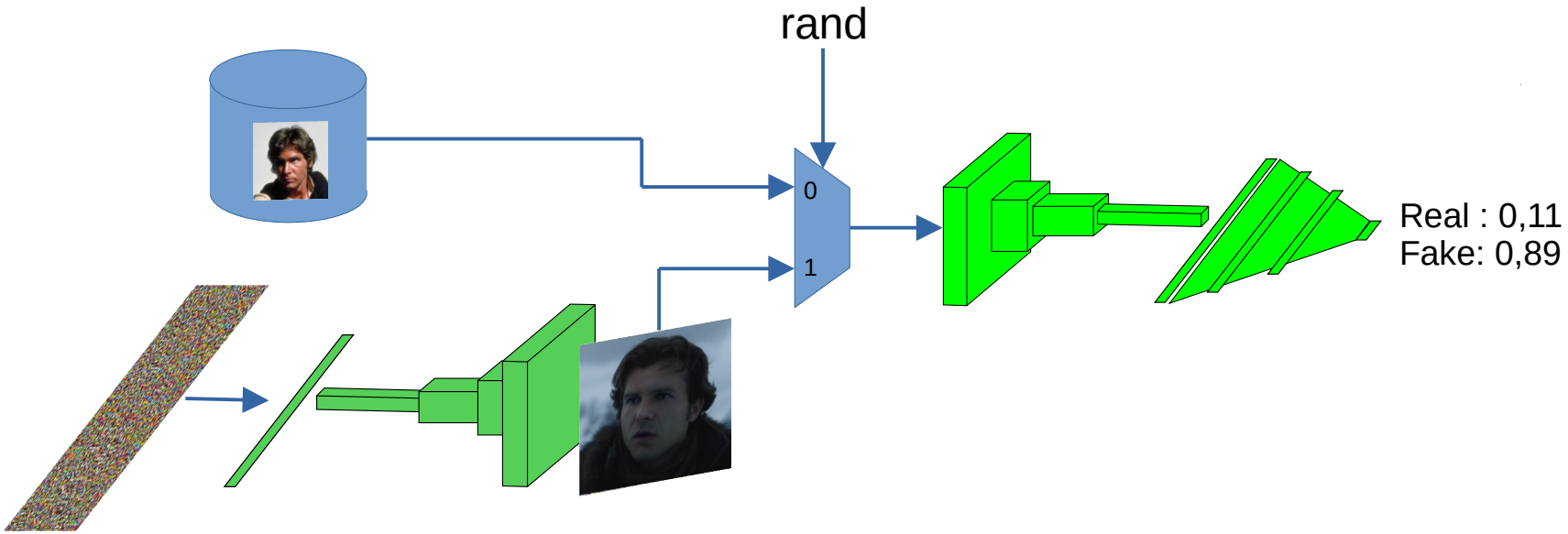
Artificial Intelligence for Robotics

- **GANs**
  - GANS to improve auroencoders
    - A generative autoencoder
    - A dataset of real images (or of the type we want to generate)
    - A discriminator network learning to distinguish real and generated images



Artificial Intelligence for Robotics

- **GANs**
  - GANS can generate novel images (or sound, texts...)
    - A decoder only
    - A random noise generator
    - A dataset of real images (or of the type we want to generate)
    - A discriminator network learning to distinguish real and generated images

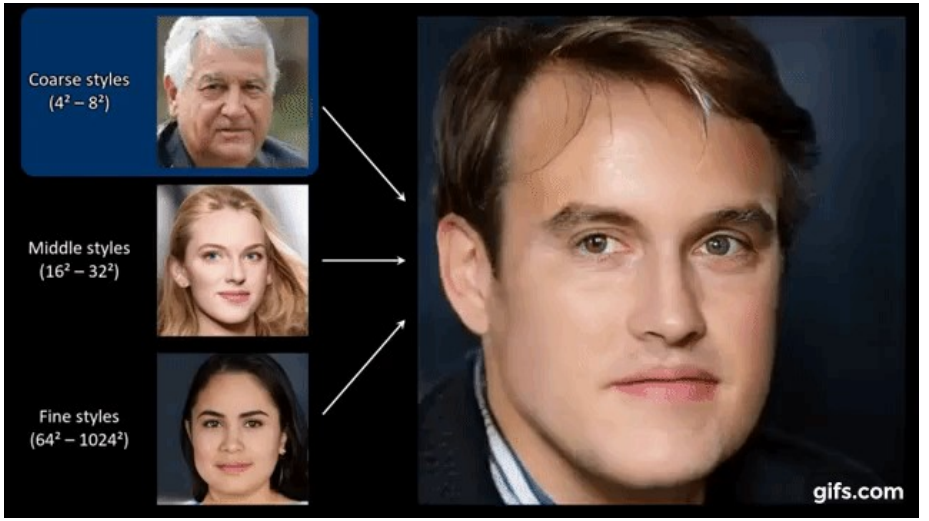


    - Any input vector can generate an image → ***latent space*** of possible vectors

Artificial Intelligence for Robotics

- **GANs**
  - GANs: latent space
    - Once trained, we can collect several representative faces with input vector

Unsupervised Representation Learning with Deep Convolutional Generative Adversarial Networks



- We can then interpolate between two points → latent space is continuous
- Or add or subtract vectors to change several parameters

Artificial Intelligence for Robotics

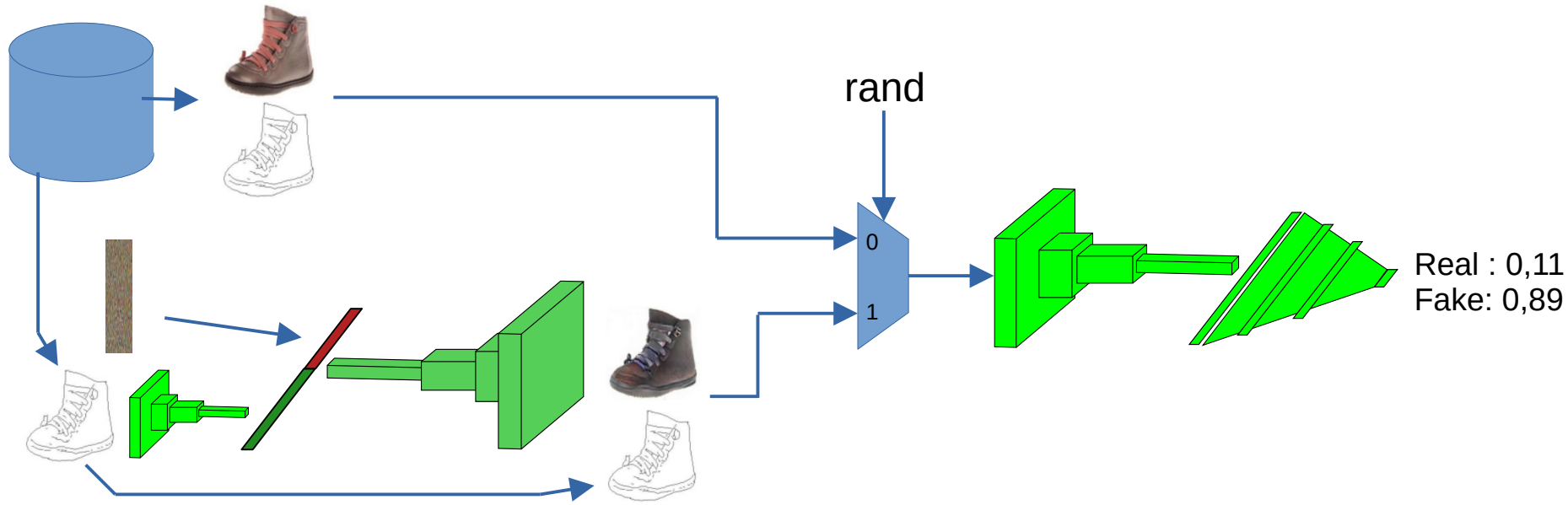
- **GANs**

Fast evolution

Generative Adversarial Networks (GANs): An Overview of Theoretical Model, Evaluation Metrics, and Recent Developments Preprint, Pegah et al., 2020

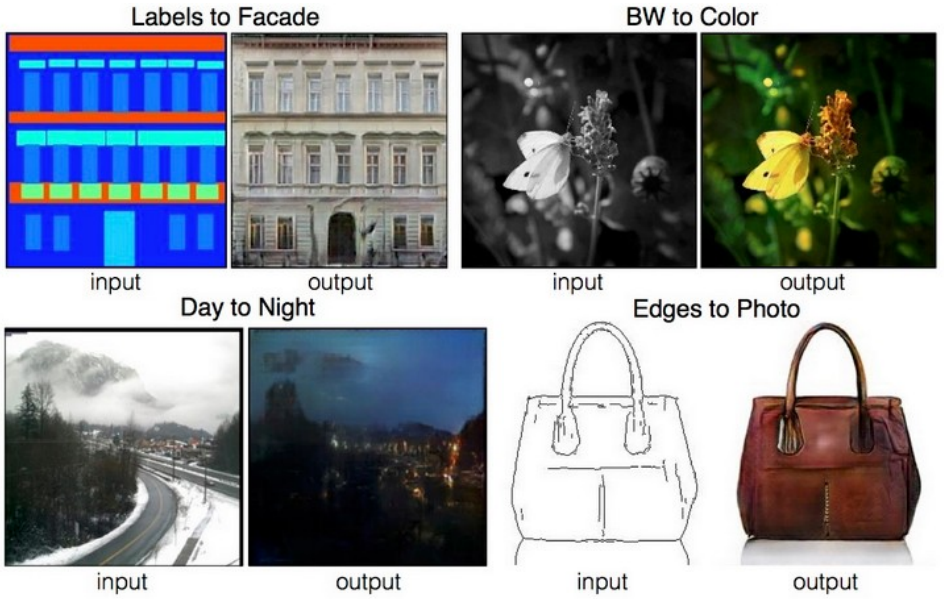
Artificial Intelligence for Robotics

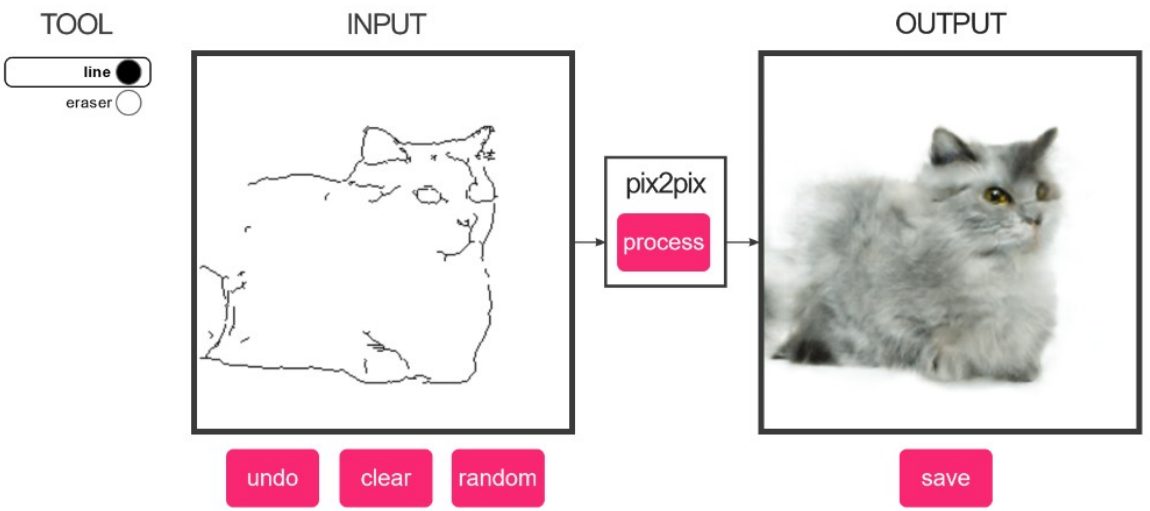
- **‘improved’ GANs**
  - Conditional GANs: add constraints on generated images
    - Add additional data to training images (e.g. drawing, numerical values...)
    - The discriminator integrates the image/information correlation
    - The generator must take into account the additional information



Artificial Intelligence for Robotics

- **‘improved’ GANs**
  - Conditional GANs :
    - Pix2pix
    - Edges2cats
    - Edges2shoes
    - Edges2handbag





https://phillipi.github.io/pix2pix/

https://affinelayer.com/pixsrv/index.html

Artificial Intelligence for robotics

Part 9 : Large Language Models and multi-modal models

Simon Gay

Artificial Intelligence for Robotics

- **Language models: how encoding a text ?**
  - The idea is to convert each word, number and punctuation into a one-hot vector (vector of 0 with one 1)

    - Thus:

      ‘a’ = [1, 0, 0, … , 0, 0, 0, …, 0, 0, 0]

      ‘dog’ = [0, 0, 0, … , 1, 0, 0, …, 0, 0, 0]

      ‘running’= [0, 0, 0, … , 0, 0, 1, …, 0, 0, 0]

  - A text is converted into a matrix
  - Problem: the length of vector is the size of your dictionary, including conjugations, multiple languages…
  - We have to reduce the size of the dictionary

Artificial Intelligence for Robotics

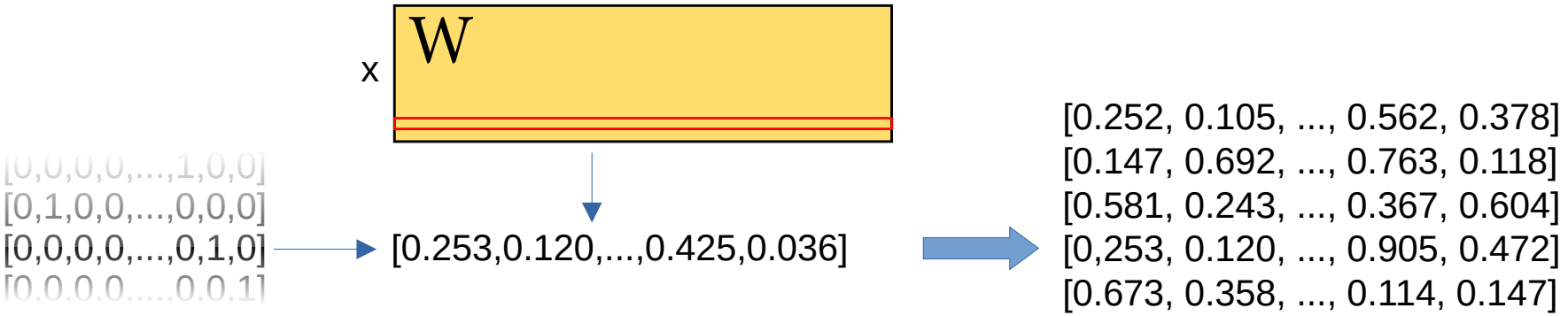
- **Language models: the tokens**
  - Many words can be split: common root, suffix, prefix…
    - playing → play-ing, played → play-ed
  - The list of tokens starts with alphabet letters, numbers and punctuation:
    - {‘a’, ’b’, ’c’,…,’A’, ’B’, ’C’,…,’1’, ’2’, ’3’,…,’.’, ’,’ ,’;’, ‘?’, ‘!’, ‘[EOS]’ }
  - A corpus of text is analyzed: when a sequence of two token appears frequently, a new token is added, until filling the desired vocabulary size
    - ‘a’,’n’ → ‘an’
    - ‘t’,’h’ → ‘th’
    - ‘_th’,’e’ → ‘_the’
  - At the end, the dictionary of token may content common words, prefix end suffix
    - {‘a’,’b’,…,’an’,’ing’,’ed’,’_pre’,…,’_the’,’_it’,’_you’,...’_dog’,’_cat’,…,[EOS]’ }

Artificial Intelligence for Robotics

- **Language models: the tokens**
  - By using tokens, the size of one-hot vectors can be significantly reduced

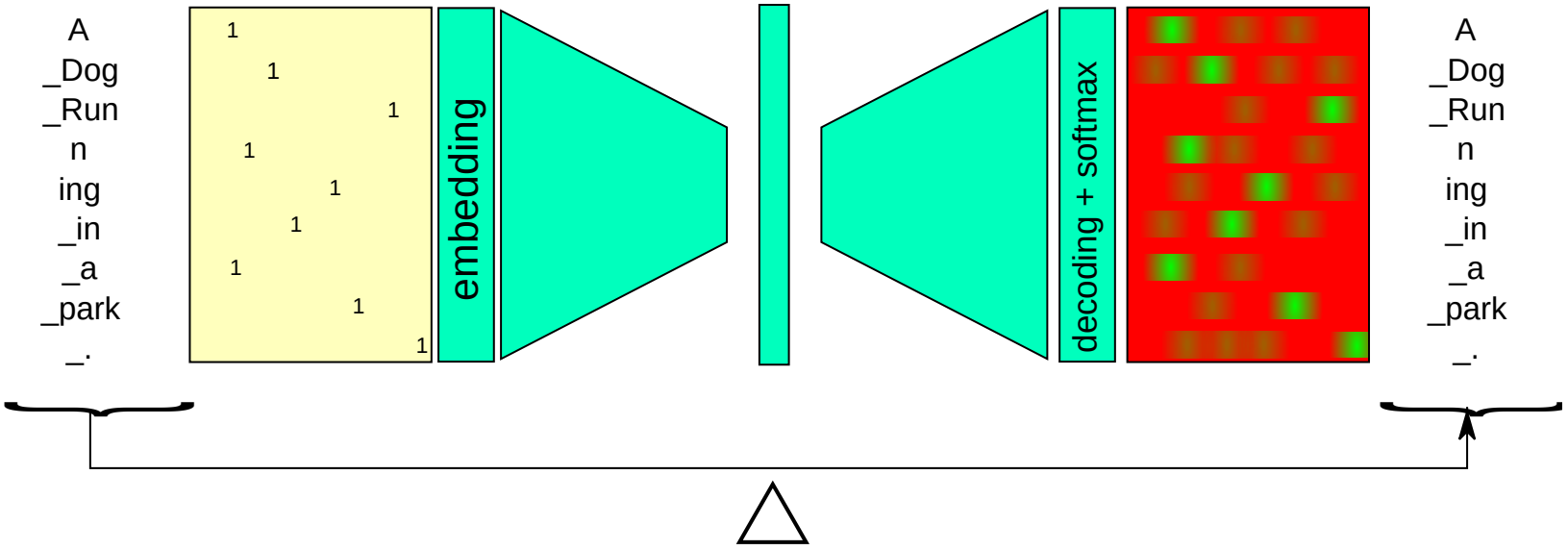
    ‘a’ = [1, 0, 0, … , 0, 0, 0, …, 0, 0, 0]
    ‘_dog’ = [0, 0, 0, … , 0, 0, 0, …, 1, 0, 0]
    ‘_run’ = [0, 0, 0, … , 0, 0, 1, …, 0, 0, 0]
    ‘n’ = [0, 0, 1, … , 0, 0, 0, …, 0, 0, 0]
    ‘ing’ = [0, 0, 0, … , 0, 1, 0, …, 0, 0, 0]

  - This matrix can then be used as the input of a network
  - Usually, one-hot vectors are converted into vectors of real values: this process is called ***embedding***. The embedded vector are smaller (4096 is a common size). The process uses a matrix of weights W (note: with one-hots, it works like a look-up table)



Artificial Intelligence for Robotics

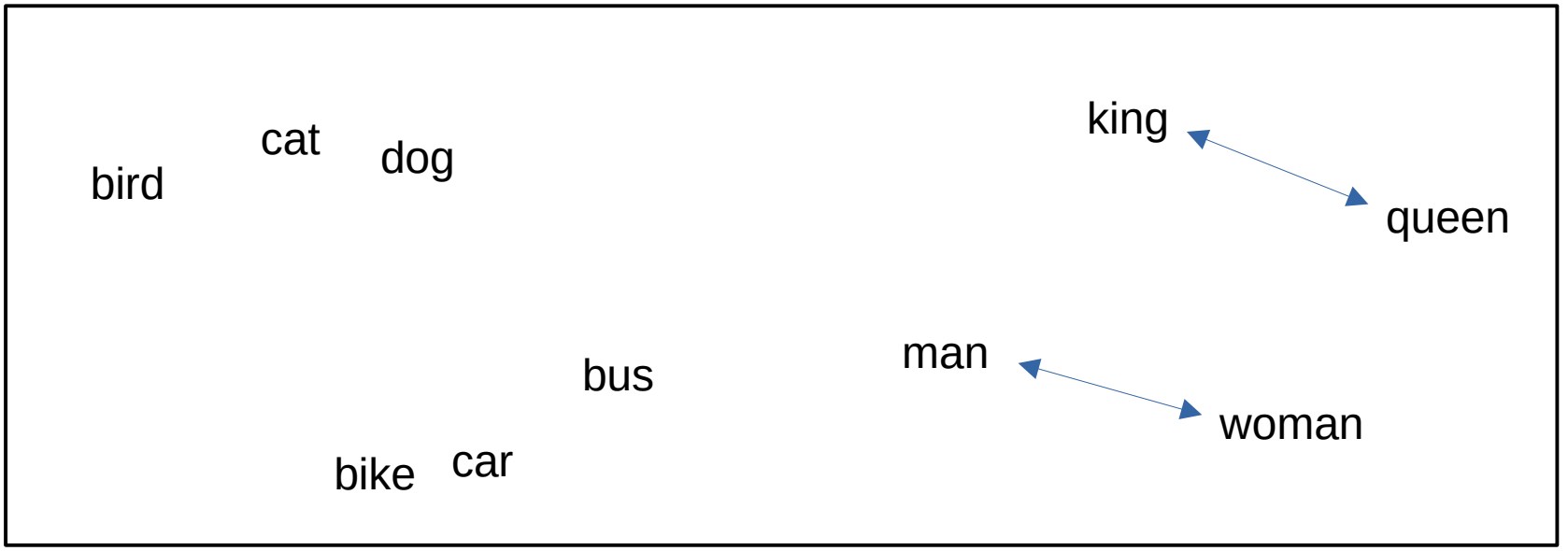
- **Language models: autoencoders**
  - The token matrix can be used, as an example, to train an autoencoder. With text, fully-connected layers are most commonly used



  - Like for image, autoencoders define a latent space for sequences of words.
  - Some models try to encourage the latent space to adopt a more structured organization, so that nearby points tend to produce similar outputs (which make interpolations more consistent): these models are called VAE (Variational AutoEncoder).
  - The models BERT and BART are trained to guess a missing token in sentence

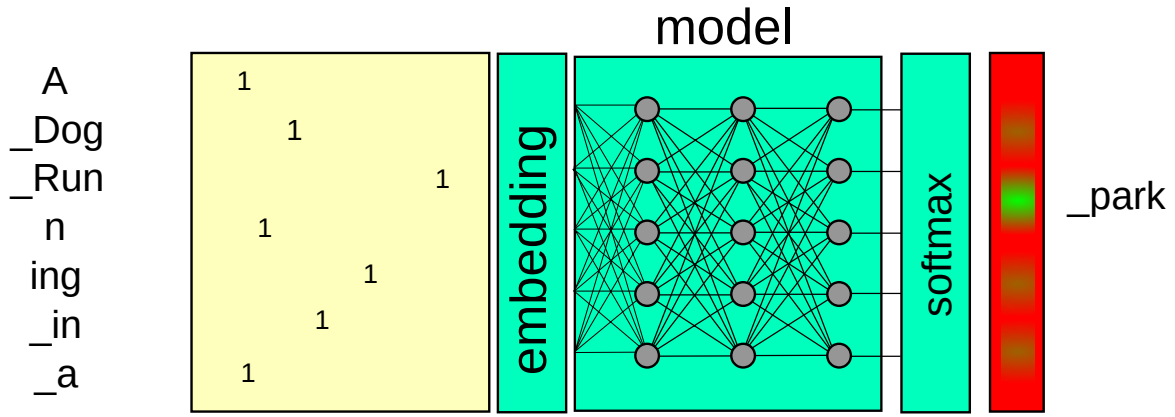
Artificial Intelligence for Robotics

- **Language models: autoencoders**
  - The embedding process also generates, during the network training, a continuous space for tokens
  - Unlike latent space that encodes full sentences (and potentially relations between the meaning of sentences), this embedded space encodes single tokens.
  - After training, and depending on the dataset, some relations may emerge:
    - Words with close meaning can be represented with close vectors
    - Differences between two pairs of related words may be similar



Artificial Intelligence for Robotics

- **Language models: generating text**
  - To generate a text, it is not needed to reconstruct the whole sentence, but just define the next token, then use the new sentence recursively until token [EOS]
    - We want to estimate P(x_t | x_1, x_2, … x_{t-1})

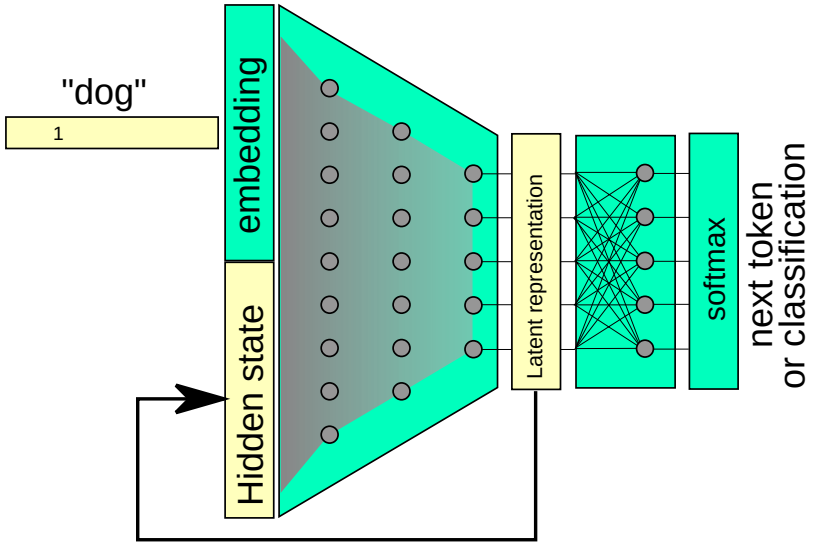


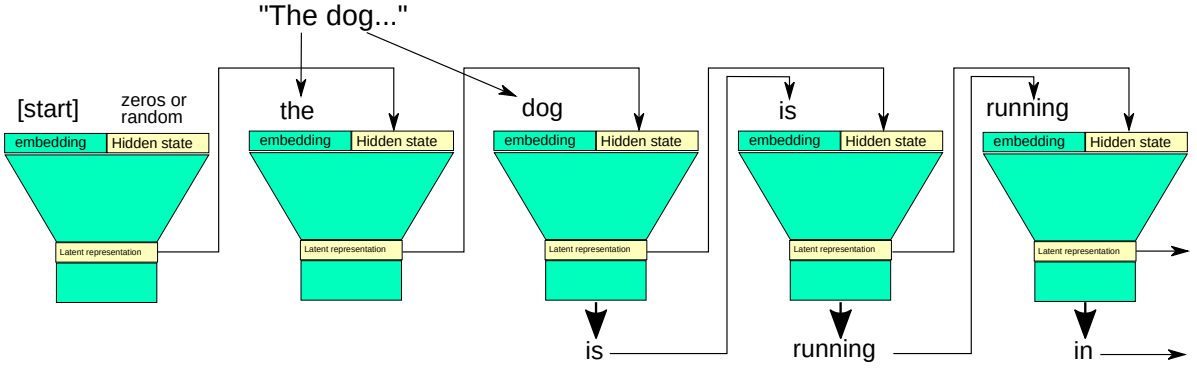
"The cat sleep" → [model] → "on"
"The cat sleep on" → [model] → "a"
"The cat sleep on a" → [model] → "bed"
"The cat sleep on a bed" → [model] → "."
"The cat sleep on a bed." → [model] → "[EOS]"

- These models are called ***autoregressive***

Artificial Intelligence for Robotics

- **Language models: Recursive Neural Networks (RNN)**
  - The network uses as input an embedded token and a second vector. This vector, called ‘hidden state’, is the output of the network on previous step.
  - The network is processed for each token of the sentence
    - The hidden state vector acts like a memory and characterize the meaning of the sequence of previous tokens
  - Additional fully-connected layers uses the hidden state vector (or the whole sequence) to predict next token, or for a classification task





  - Limitation: the whole sequence of previous token is embedded in a single hidden state vector, information can thus be diluted

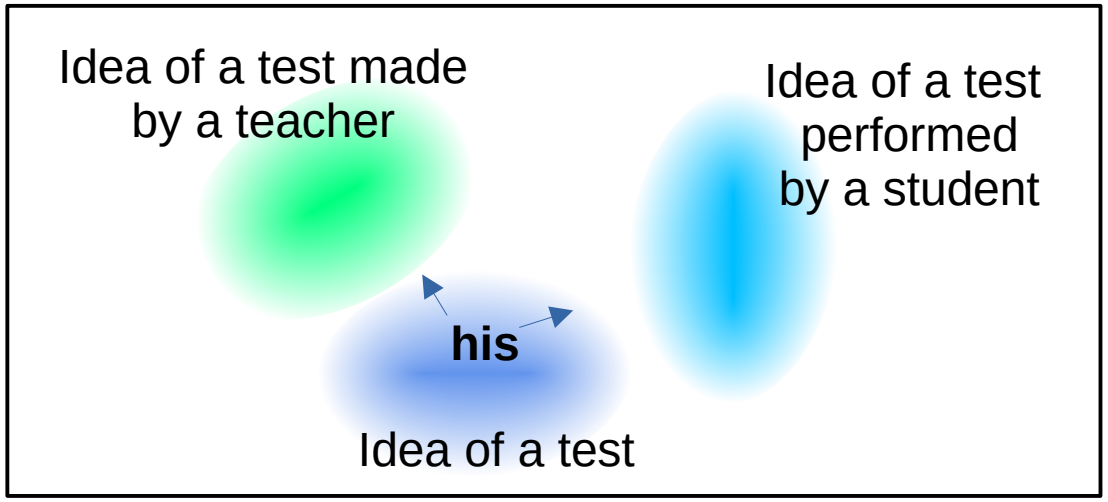
Artificial Intelligence for Robotics

- **Language models: The transformer**
  - Architecture created in 2017
  - sequence-to-sequence (e.g. translation)
  - GPT is a decoder-only transformer (text generation)
  - Based on **Transformer Blocks**:
    - The aim is to make embedded vectors more contextualized
    - Two steps:
      - Attention: the token is modified according to other tokens
      - Feed forward : each token is modified independently, before being passed to the next block.

Artificial Intelligence for Robotics

- **Language models: Principle of *Attention***
  - Contextualize a token using other tokens of the sentence

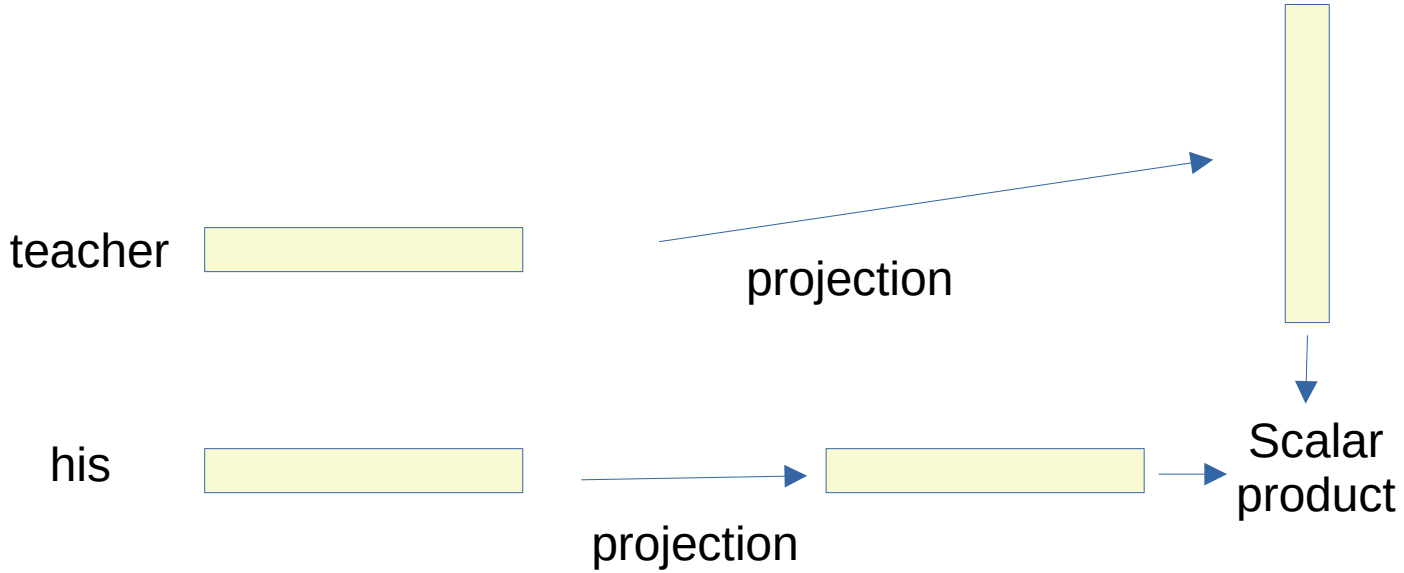
    "The teacher wanted to meet the student about **his** test"
    - "his" can refers to token "teacher" or token "student"
    - Two contexts, thus two positions in latent space



    - The "change" of the contextualized token depends on model training
      - Which sentences were most represented in dataset ?

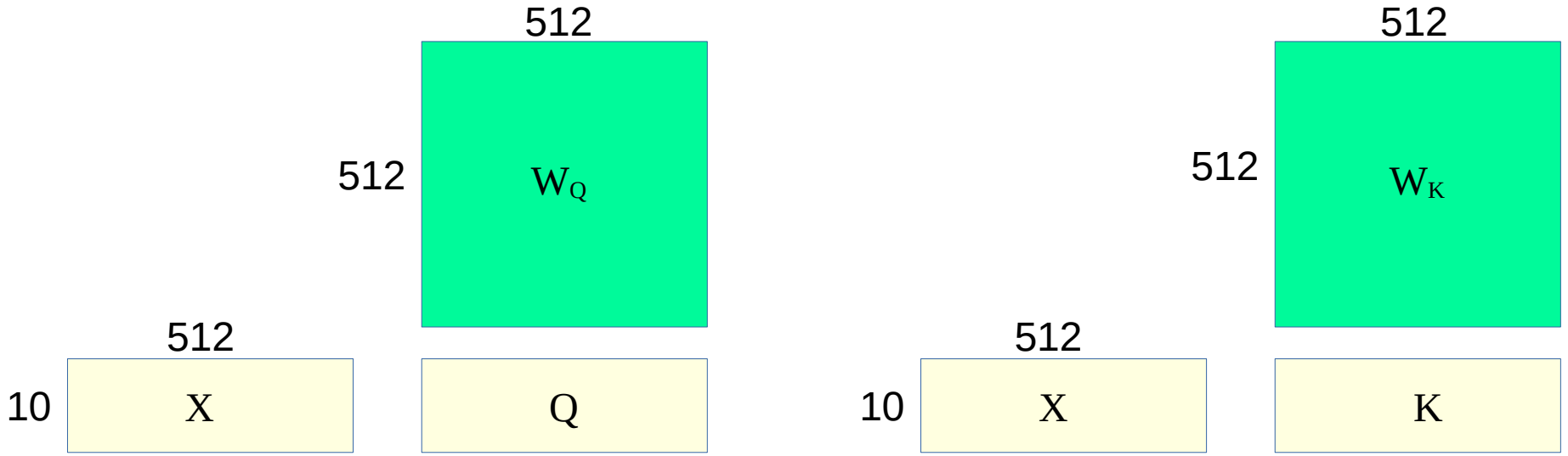
Artificial Intelligence for Robotics

- **Language models: Principle of *Attention***
  - The idea is to compare each token with each other and define how the second token is important to understand the context of the former
    - First, the two tokens are projected to another latent space where the comparison is meaningful
    - The ‘score’ is then given by they scalar product



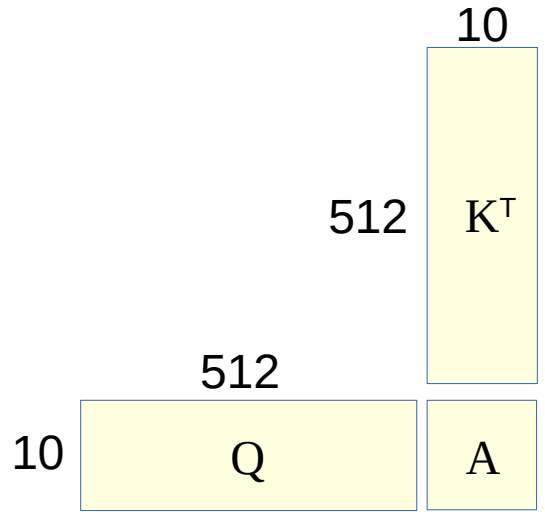
Artificial Intelligence for Robotics

- **Language models: Principle of *Attention***
  - The model uses two matrices W_Q and W_K for this reference change
    - For this example, we have 10 tokens embedded on d_e=512 parameters. The matrices have a dimension of attention d_k=512
    - We compute the query and key matrices Q and K
      - Q gives the projected vectors of tokens to contextualize
      - K gives the projected vectors of tokens being matched against.



Artificial Intelligence for Robotics

- **Language models: Principle of *Attention***
  - A matrix product is performed with matrices Q and K to provide the attention matrix A: each value A_{ij} gives the importance of token j to contextualize token i
  - The values are normalized, a SoftMax is performed on each line

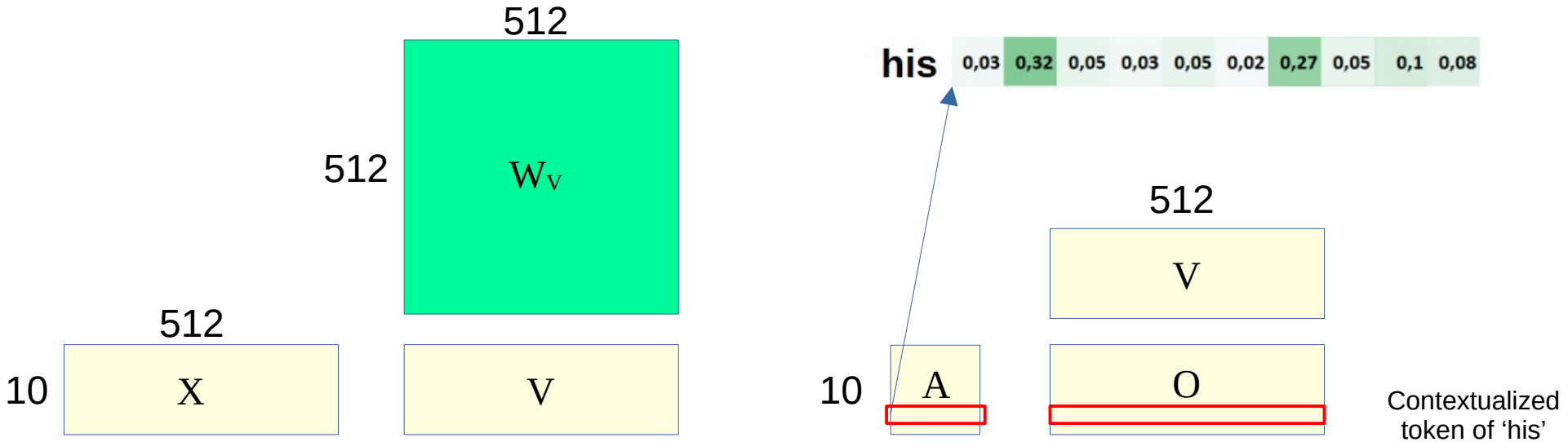


$$A = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right)$$

A	the	teacher	wanted	to	meet	the	student	about	**his**	test
the	0,25	0,15	0,1	0,05	0,05	0,15	0,05	0,05	0,05	0,1
teacher	0,1	0,25	0,1	0,05	0,1	0,05	0,1	0,05	0,1	0,1
wanted	0,05	0,25	0,2	0,1	0,15	0,05	0,05	0,05	0,05	0,05
to	0,05	0,1	0,2	0,15	0,25	0,05	0,05	0,05	0,05	0,1
meet	0,05	0,15	0,15	0,1	0,2	0,05	0,2	0,05	0,03	0,02
the	0,05	0,1	0,05	0,05	0,1	0,25	0,25	0,05	0,05	0,05
student	0,05	0,1	0,05	0,05	0,15	0,1	0,25	0,05	0,1	0,1
about	0,05	0,1	0,05	0,05	0,1	0,05	0,2	0,2	0,1	0,1
his	**0,03**	**0,32**	**0,05**	**0,03**	**0,05**	**0,02**	**0,27**	**0,05**	**0,1**	**0,08**
test	0,05	0,15	0,05	0,05	0,1	0,05	0,25	0,1	0,15	0,05

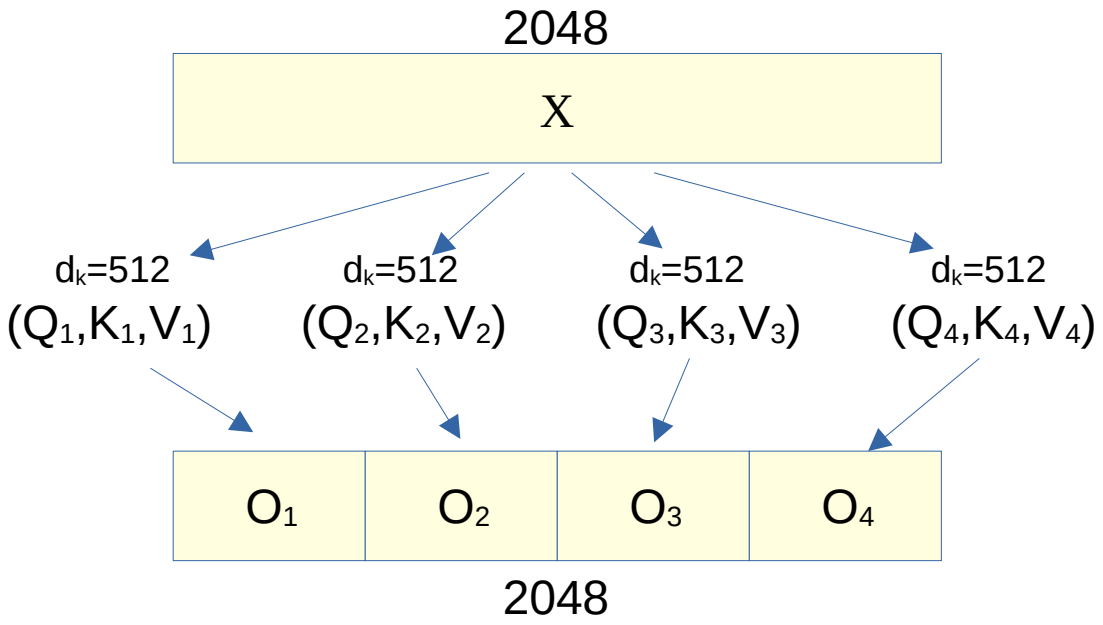
Artificial Intelligence for Robotics

- **Language models: Principle of *Attention***
  - Now, we will recompose the token's new vector using information from all tokens, weighted according to their matching score.
  - Tokens are first projected in another latent space, encoding more the information that can be retrieved, using matrix of weights W_V
  - By performing A.V, the result is a matrix O where each line is a sum of transformed token V_j weighted by its matching with token I
  - Each line of O is a transformed token with more contextual information



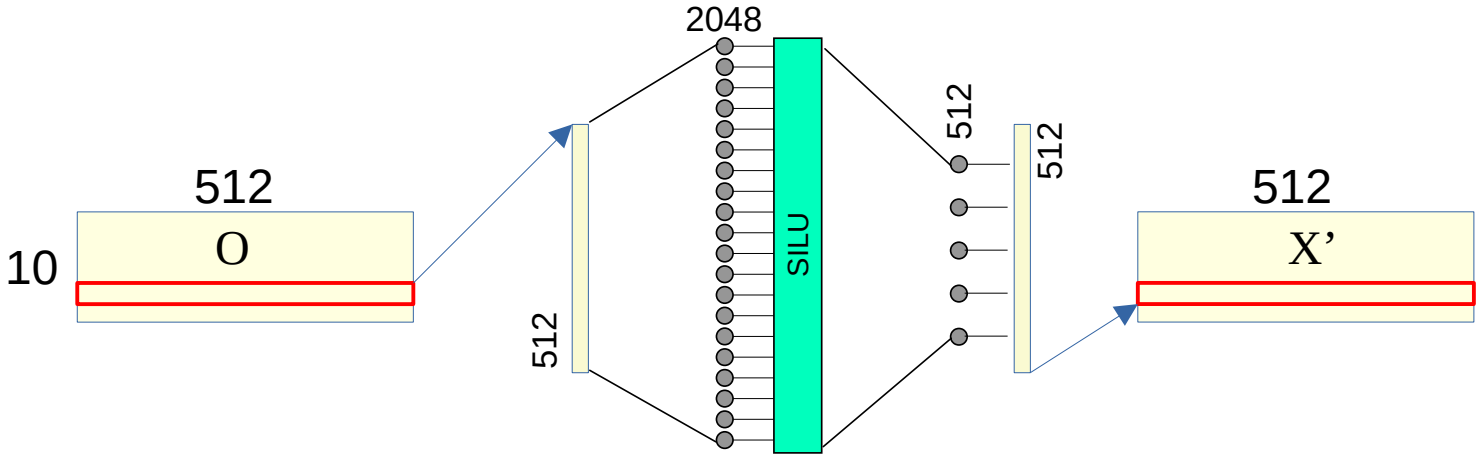
Artificial Intelligence for Robotics

- **Language models: Principle of *Attention***
  - In practice, transformer blocks uses multiple triplets (W_Q,W_K,W_V) learned in parallel and independently, generating multiple ‘interpretations’ for the same context. Each triplet is called **Head**
  - Resulting matrices are concatenated to form result O
    - This is why $d_e = nb_{heads} \times d_a$



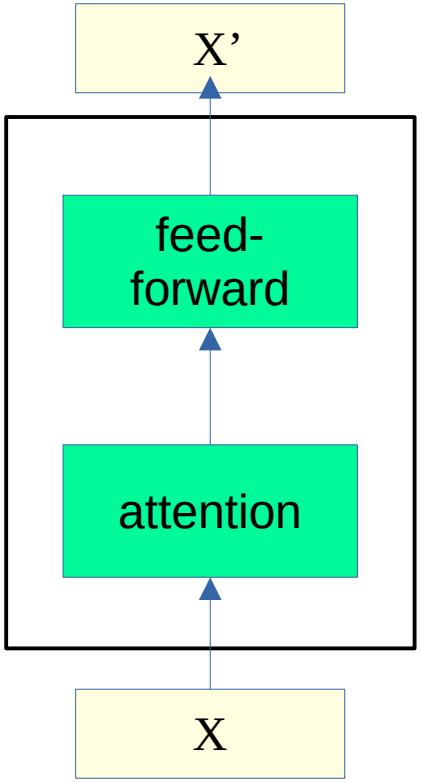
Artificial Intelligence for Robotics

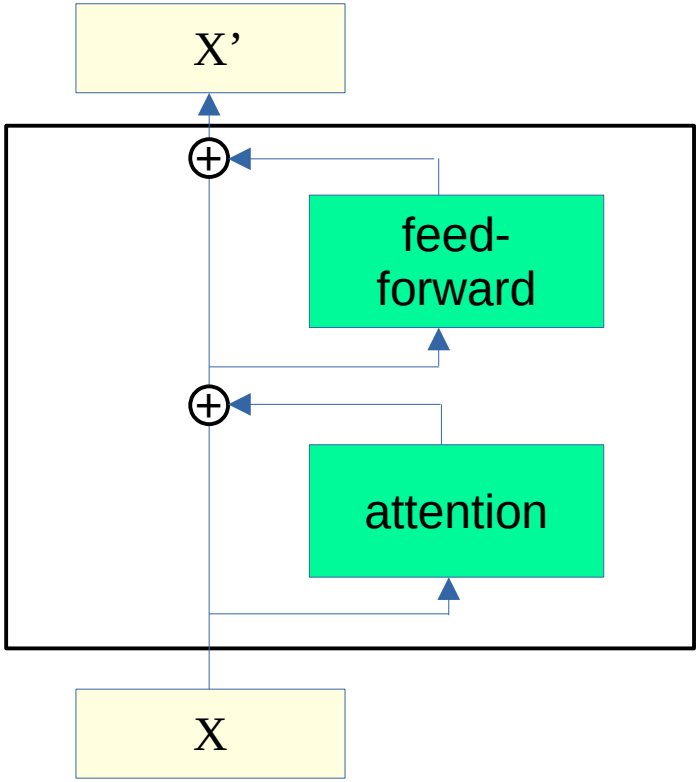
- **Language models: Principle of *Feed forward***
  - The vectors in Attention's output is more like a vector of features
  - The feed forward is a fully-connected network converting the 'features' into a 'true' contextualized embedded and contextualized token, through a non-linear function
  - Usually:
    - Fully-connected layer with large number of neurons (>d_e)
    - Activation layer (mostly GELU or SiLU)
    - Fully-connected layer of size d_e
  - The same network is applied on <u>each</u> token <u>independently</u>



Artificial Intelligence for Robotics

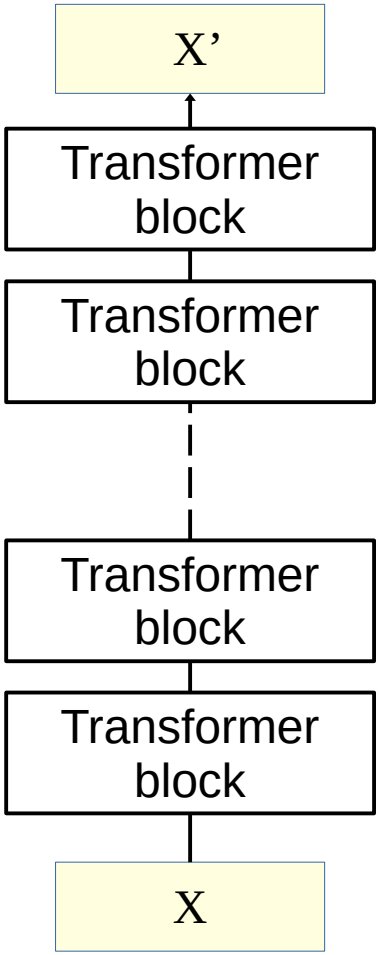
- **Language models: Principle of *residual connection (skip connection)***
  - Instead of learning the output, why not just learn the correction ?
  - At the beginning of training, values might be low, thus, the output is closed from input : X ≈ X'





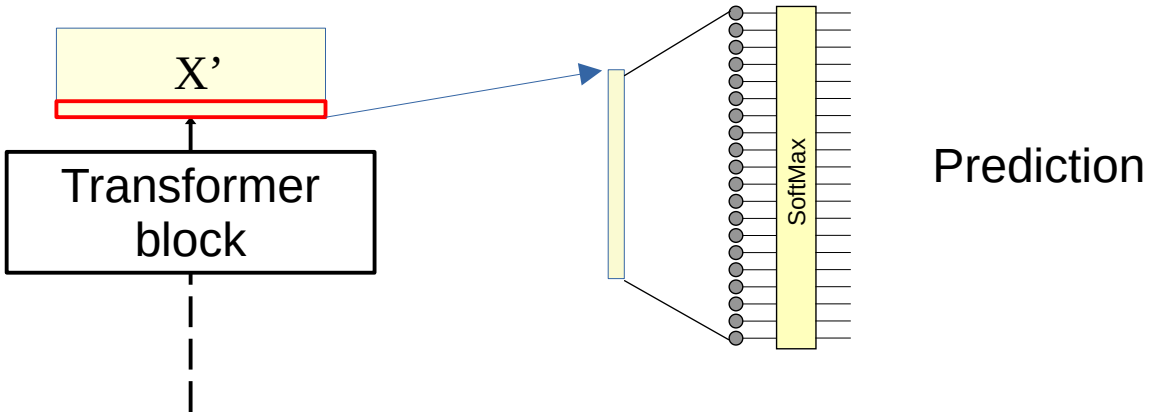
Artificial Intelligence for Robotics

- **Language models: Large Languages Models**
  - Transformer blocks can then be chained
    - Current models can contains billions of parameters



Artificial Intelligence for Robotics

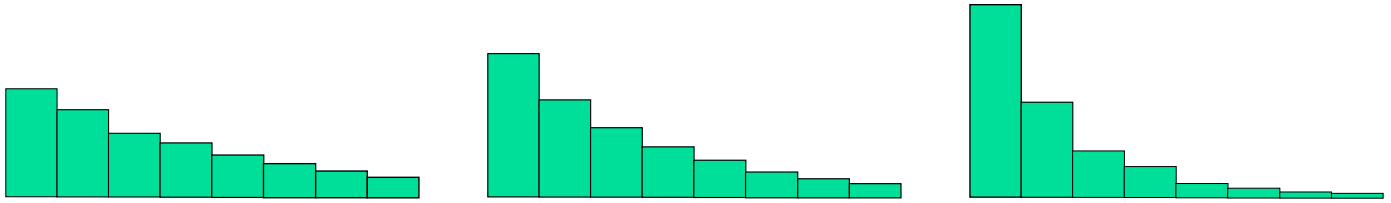
- **Language models: Predicting next token**
  - The last embedded token matrix contains vectors characterizing highly-contextualized tokens. The last one might contains the meaning of the whole sequence of words. LLMs uses the last token for prediction
  - A fully-connected layer with softmax activation function with the size of the token dictionary is connected to this last token, giving the probability for each token.
    - In many models, the weights of this layer is the same matrix W than for embedding process (weight tying)



Artificial Intelligence for Robotics

- **Language models: Predicting next token**
  - The next token is randomly chosen, with probability of selection given by the corresponding value in the softmax output vector.
  - The probability distribution can be modified with **temperature**
  - Output vector (before softmax), is divided by a constant T
    - $P_i = \text{softmax}(x_i' / T)$
  - With T > 0, probability differences are amplified → more deterministic results
  - With T < 0, probability differences are reduced → more diverse results

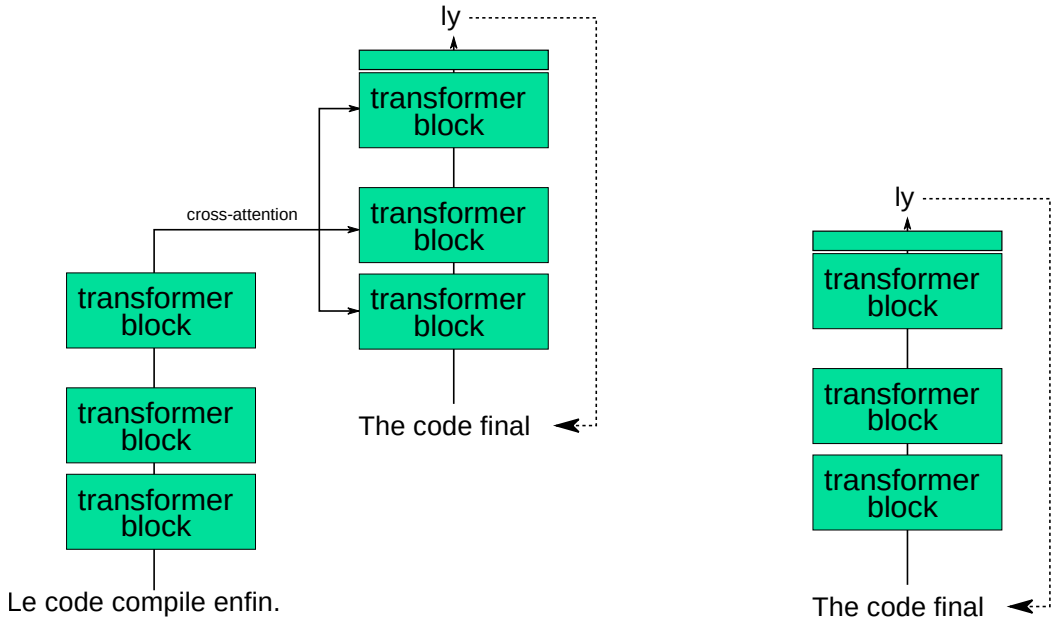
Distribution of probabilities



T < 1 T = 1 T > 1

Artificial Intelligence for Robotics

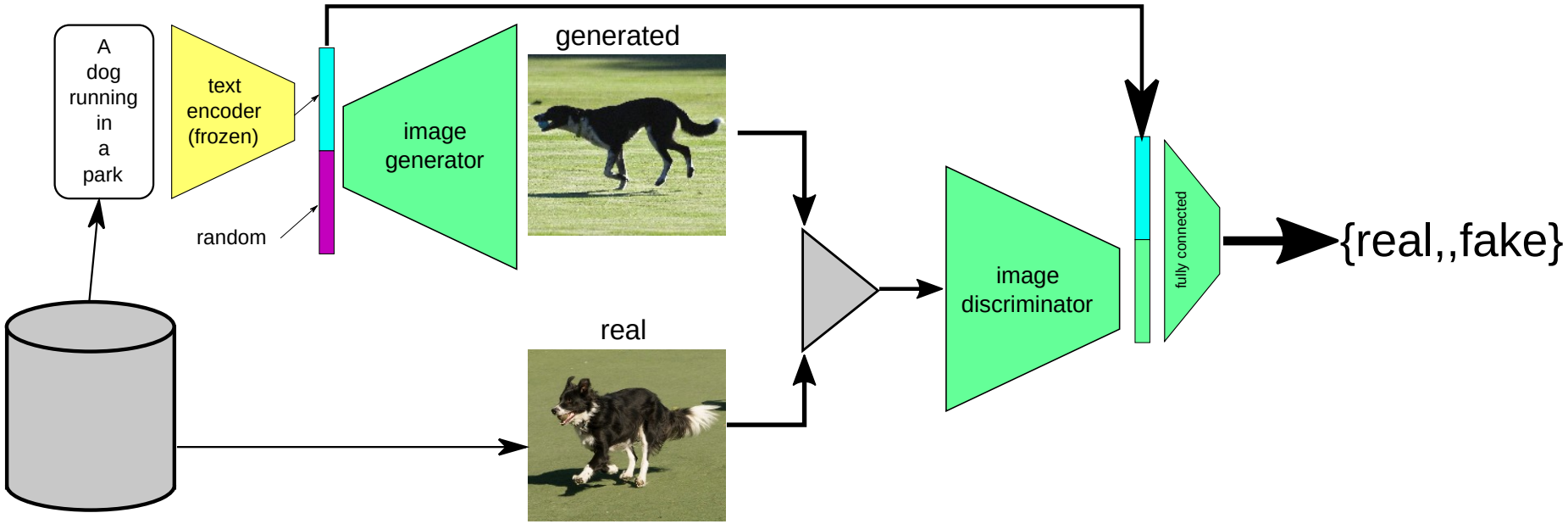
- **Language models: Transformer architecture and GPT**
  - The original transformer architecture is a sequence-to-sequence model (e.g. translation). A sub-model encode a sentence (encoder), and send contextualized tokens in attention blocs of a second sub-model constructing the new sentence (decoder)
    - An attention mechanism integrating information from other source is called ‘cross-attention’ (as opposed to self-attention)
  - GPT is a decoder-only model, constructing new sentence from the beginning of a sentence, with self-attention



Multi-modal models: combining texts and images

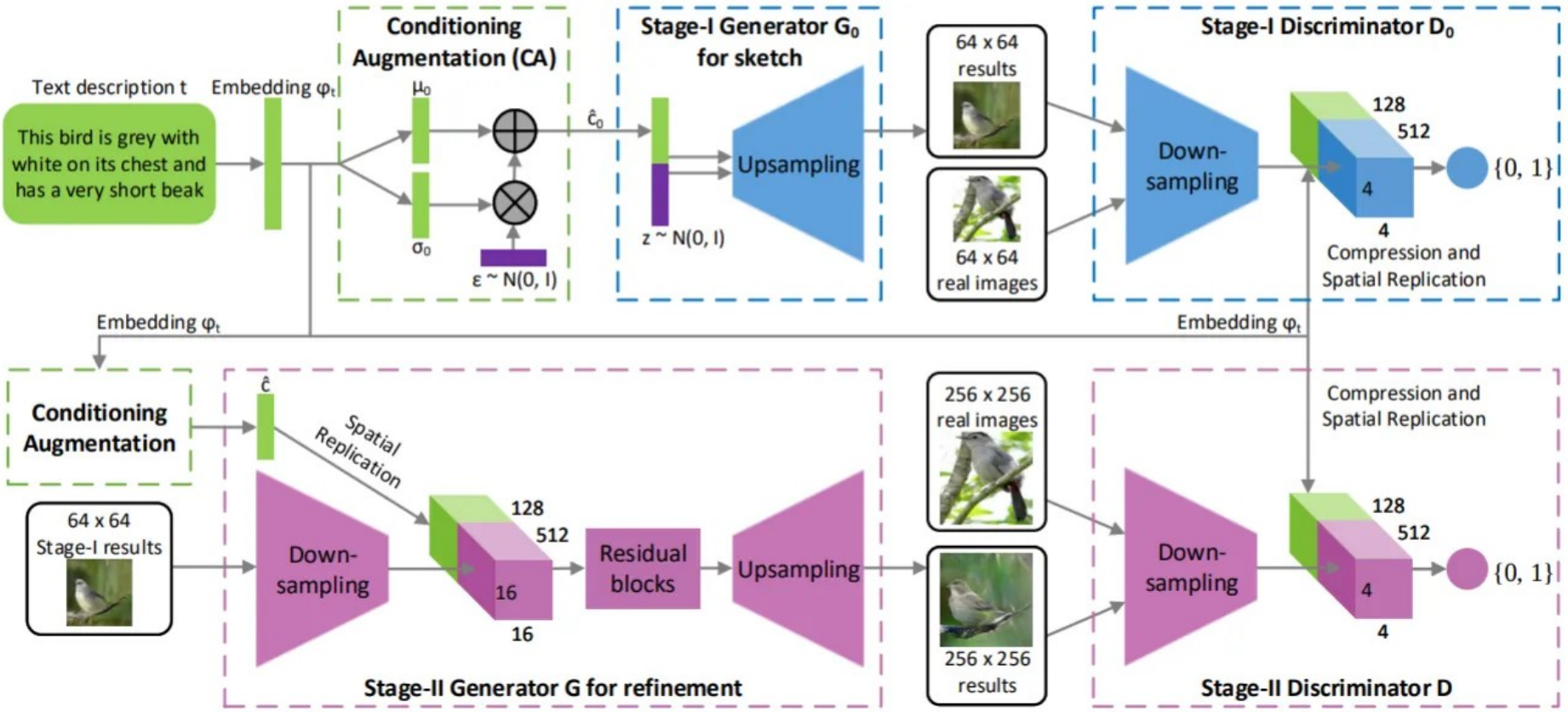
Artificial Intelligence for Robotics

- **Text2image: The Conditional GAN approach (Reed et al. 2016)**
  - Pre-trained RNN text encoder: condition vector
  - The Conditional GAN uses a random vector (‘seed’) and condition vector
  - The discriminator and generator can use relation between images and text



Artificial Intelligence for Robotics

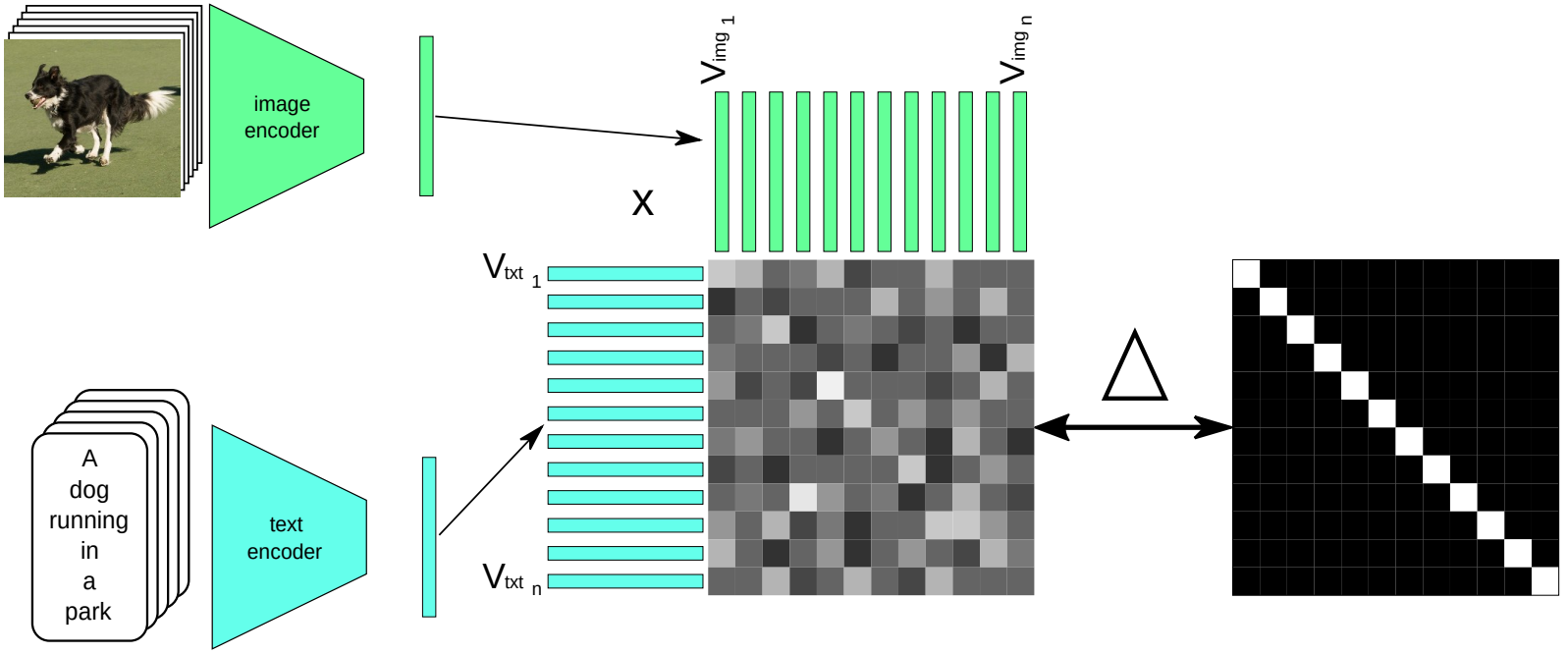
- **Text2image : The Conditional GAN approach (StackGAN 2017)**
  - StackGAN[1] uses a second ‘stage’ with an autoencodeur with condition to increase the resolution of generated images



[1]https://arxiv.org/abs/1612.03242

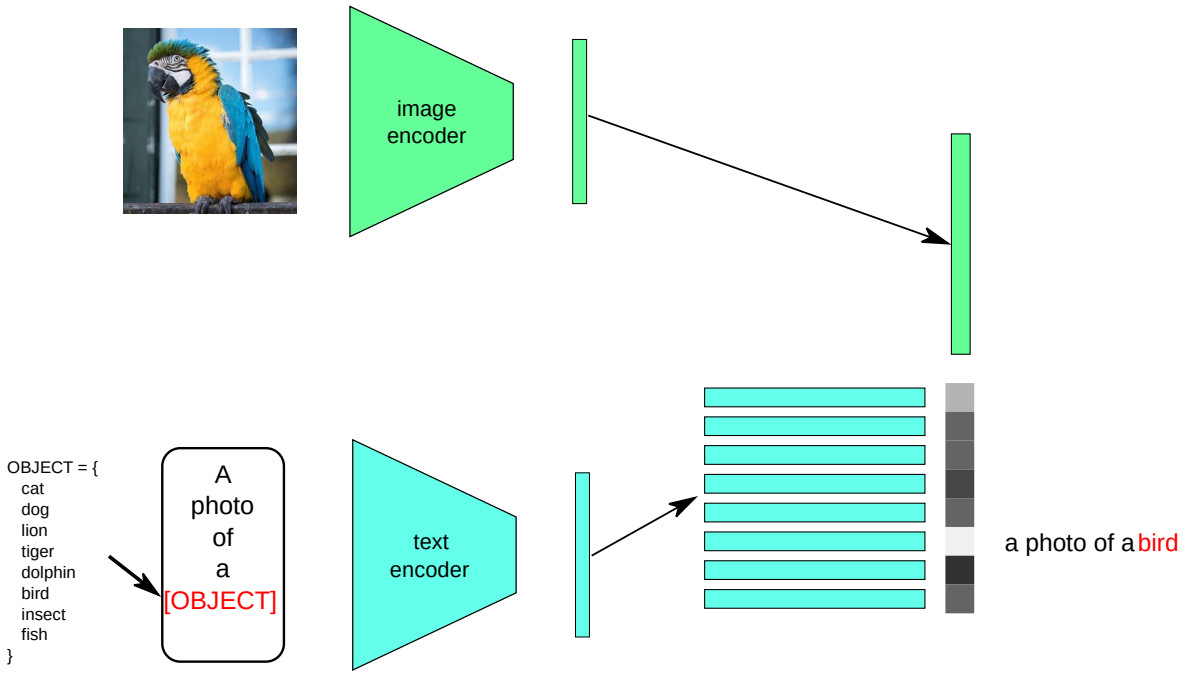
Artificial Intelligence for Robotics

- **Link models: CLIP (Contrastive Language–Image Pre-training, 2021)**
  - A text encoder (transformer) and an image encoder (CNN and later vision transformer)
  - Correlation matrix: maximizing diagonal and minimizing the others
  - Idea: force latent spaces of the two encoders to converge toward similar distribution: the sentence ‘a dog running in a park’ will be represented with a similar vector than an image of such a scene.



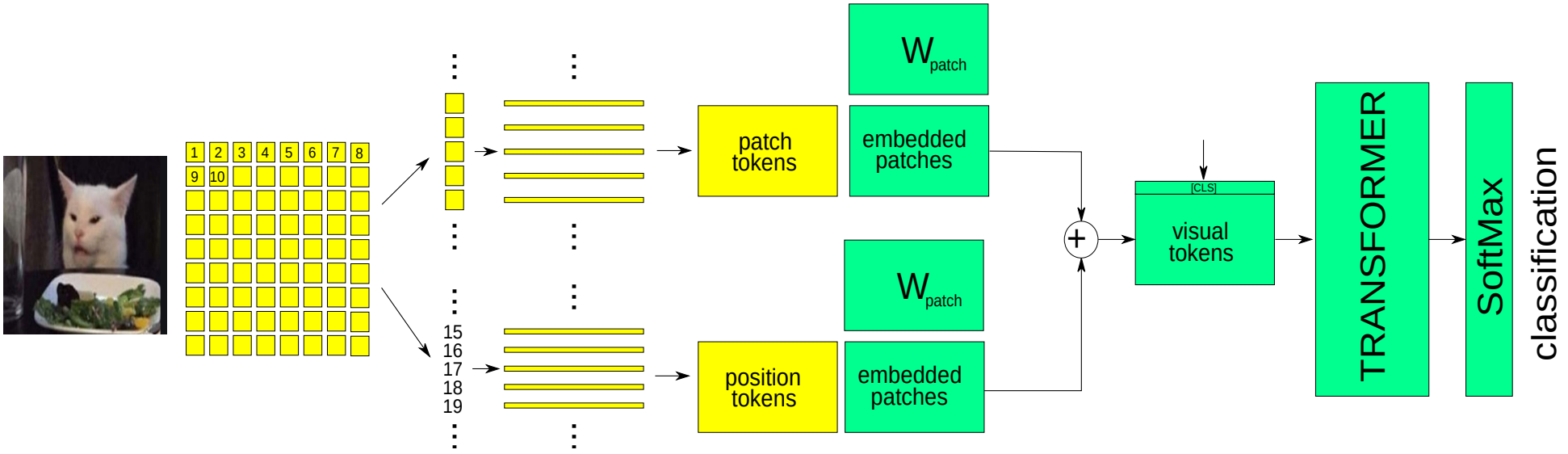
Artificial Intelligence for Robotics

- **Link models: zero-shot prediction !**
  - Create a set of sentences with replaceable word
  - Present an image to the network and get latent vector
  - Test categories: estimation of the category with best matching
  - Classification *a posteriori* ! (Zero-Shot prediction)



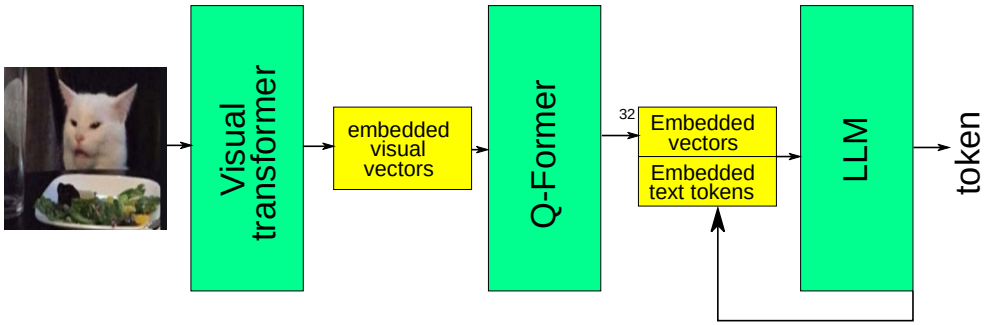
Artificial Intelligence for Robotics

- **Visual transformer**
  - Split the image into small patches (for example 16x16) with an id (position)
  - Flatten the patches, then perform an embedding (like for tokens)
  - Also embed the position (still like a token, similar to a look-up table))
  - Add patch embedding and positional embedding vectors
  - The list of vector forms a matrix X used as input for a transformer block
  - Particularity: an additional trained vector ***[CLS]*** is added at the beginning of embedded tokens. This vector is expected, through transformer blocks, to accumulate information and is used as last output of the transformer chain.
  - The visual transfomer is trained as a classifier (encoder)

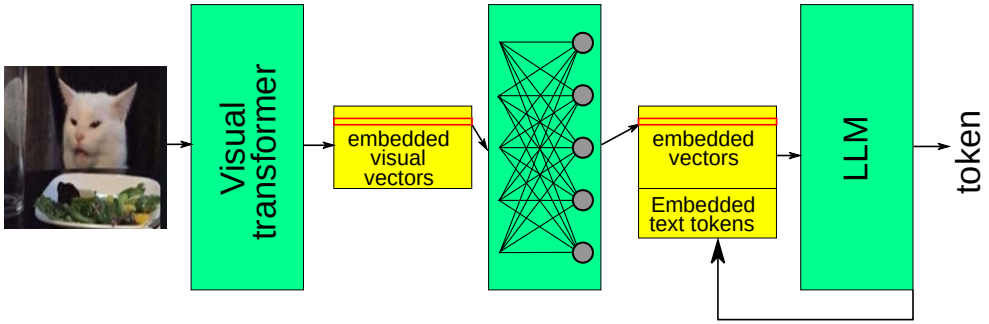


Artificial Intelligence for Robotics

- **True multi-modal models**
  - Two approaches:
    - BLIP-2: uses a pre-trained visual encoder and a pre-trained LLM, adds a Querying Transformer (or Q-former) to ‘translate’ contextualized visual tokens into a predefined number of ‘representation’ tokens (usually 32) that can be exploited by the LLM.
      - Instead of computing $Q=XW_Q$, the Q-former uses $Q=Q_0W_Q$ with Q_0 the matrix of (usually) 32 query vectors.
    - LlaVA: also uses a pre-trained visual encoder and a pre-trained LLM, but connect them through a linear layer, the LLM is then fine-tuned. The number of embedded vector is the same than the number of patches



Blip-2



LLaVa

Artificial Intelligence for Robotics

- **How can a robot use a LLM ?**
  - Planning : a LLM can generate a sequence of low-level action from a high-level command
    - A prompt indicates what the LLM is expected to do, then the high-level command is sent
  - Image/scene recognition : a visual LLM can describe a scene, present objects or a performed action
  - A script can find specific keywords in the LLM’s answer to control a robot.

This mini GPU runs LLM that controls this robot
(Nikodem Bartnik, https://www.youtube.com/watch?v=0O8RHxpkcGc)

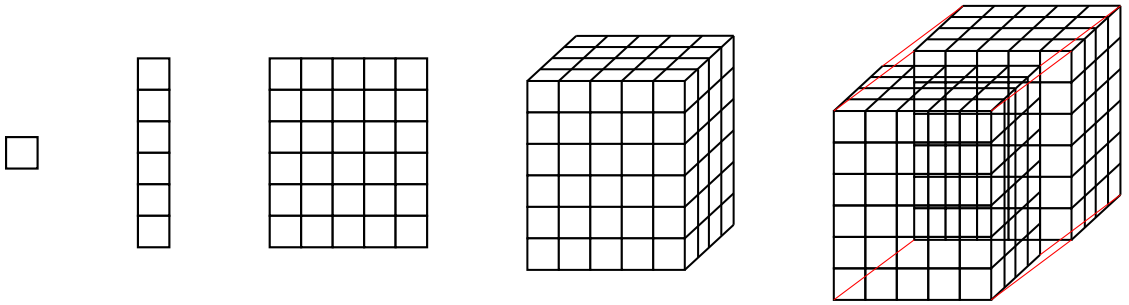
Libraries for Deep Learning

Artificial Intelligence for Robotics

- **Several libraries for developing large neural models:**
  - TensorFlow (by Google), open-source since 2015
    - Qince 2017, there is a light version for embedded systems
  - MXNet (Apache), can use multiple languages
  - Caffe
  - Theanos
  - Microsoft Cognitive Toolkit
  - PyTorch (Facebook), based on Torch
  - Keras: libraries of high-level functions for Tensorflow and Theanos

Artificial Intelligence for Robotics

- **TensorFlow**
  - Library designed for *tensor* manipulation
    - A tensor is a matrix with any number of dimensions



  - Many optimized functions
  - Possibility to run on GPU !

Artificial Intelligence for Robotics

- **TensorFlow**
  - Import : `import tensorflow as tf`
  - Constructors :

```
A = tf.constant([[1,2,3],[4,5,6]])
A = tf.Variable([[1,2,3],[4,5,6]])

A = tf.ones((3,3))
A = tf.zeros((3,3))
A = tf.eye(3)

A = tf.random.normal((3,3), mean=0, stddev=1)
A = tf.range(5)
```

```
tf.Tensor(
[[1 2 3]
 [4 5 6]], shape=(2, 3), dtype=int32)
```

```
<tf.Variable 'Variable:0' shape=(2, 3) dtype=int32, numpy=
array([[1, 2, 3],
       [4, 5, 6]], dtype=int32)>
```

```
tf.Tensor(
[[1. 1. 1.]
 [1. 1. 1.]
 [1. 1. 1.]], shape=(3, 3), dtype=float32)
```

```
tf.Tensor(
[[0. 0. 0.]
 [0. 0. 0.]
 [0. 0. 0.]], shape=(3, 3), dtype=float32)
```

```
tf.Tensor(
[[1. 0. 0.]
 [0. 1. 0.]
 [0. 0. 1.]], shape=(3, 3), dtype=float32)
```

```
tf.Tensor(
[[ 1.3420206  1.7878479 -1.0414563]
 [-1.5547394  0.5375859 -1.1487345]
 [ 1.2995738  1.029388   0.9286815]], shape=(3, 3), dtype=float32)
```

```
tf.Tensor([0 1 2 3 4], shape=(5,), dtype=int32)
```

Artificial Intelligence for Robotics

- **TensorFlow**
  - Some functions to manipulate tensors:

```
C = tf.add(A,B)          # somme elt par elt
C = tf.subtract(A,B)     # soustraction elt par elt
C = tf.multiply(A,B)     # mutliplication elt par elt
C = tf.divide(A,B)       # division elt par elt

s = tf.tensordot(A,B, axes=1) # produit scalaire
C = tf.matmul(A,B)            # produit matriciel

N = C.numpy()                 # conversion tenseur->numpy
C = tf.convert_to_tensor(N)   # conversion numpy-> tenseur

V = tf.Variable(A)            # conversion constant->variable
V.assign([5,3,9])             # mise à jour variables
```

Artificial Intelligence for Robotics

- **Keras**
  - Library of high-level functions based on tensorflow or Theanos

```
import tensorflow as tf
from tensorflow import keras
```

  - Can easily perform the following operations:
    - Load a ‘common’ dataset (mostly for benchmarks)
    - Define the network architecture
    - Define training parameters
    - Train the network
    - Evaluate ans use the network

Artificial Intelligence for Robotics

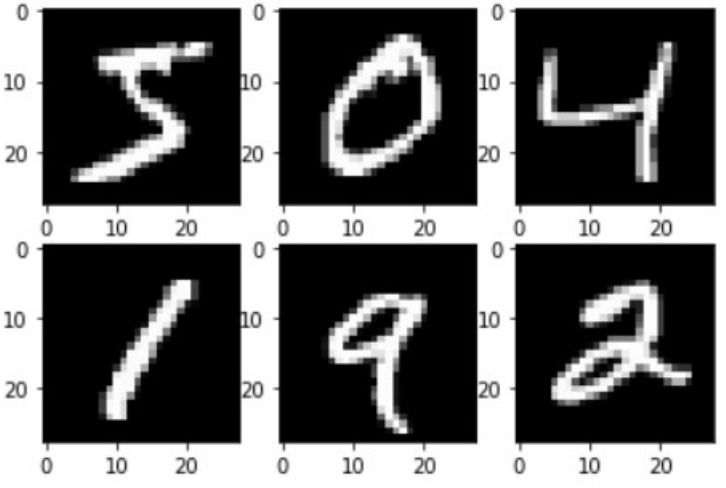
- **Keras**

  - Load a dataset:

```
dataset = keras.datasets.mnist

(img_train, output_train),(img_test,output_test) = dataset.load_data()

img_train=img_train/255.0
img_test=img_test/255.0
```

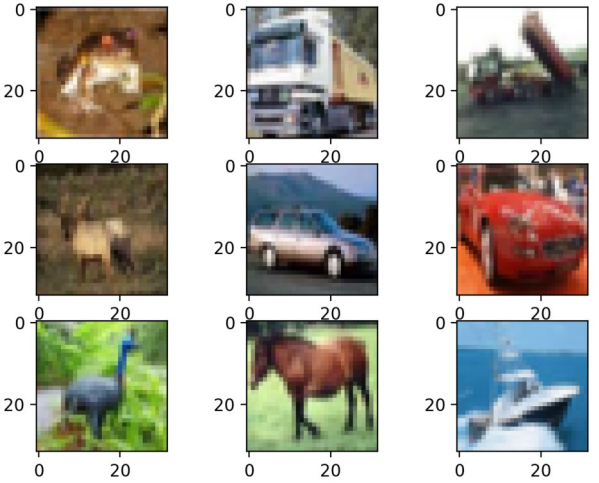
```
print(img_train.shape)
print(output_train.shape)
```

```
(60000, 28, 28)
(60000,)
```



Artificial Intelligence for Robotics

- **Keras**
  - Available Datasets:
    - Boston housing prices (numerical values)
      → *boston_housing*
    - CIFAR-10 (color images of 32x32)
      → *cifar10*
    - CIFAR-100 (same with 100 classes)
      → *cifar100*
    - Fashion MNIST (images B&W 28x28)
      → *fashion_mnist*
    - IMDB movie reviews (texts) → *imdb*
    - MNIST (images N&B 28x28) → *mnist*
    - Reuters articles (texts) → *reuters*



CIFAR-10
(50 000 images)

Artificial Intelligence for Robotics

- **Keras**
  - Define network architecture:
    - The model is defined layer by layer (as with our multi-layer)
    - Initialization of the model (sequential methof) :

```
network = keras.Sequential()
```

    - Then successively add layers:

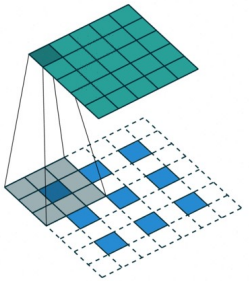
```
network.add(keras.layers.Flatten(input_shape=(28,28)))
network.add(keras.layers.Dense(40, activation='sigmoid'))
network.add(keras.layers.Dense(10, activation='sigmoid'))
```

Artificial Intelligence for Robotics

- **Keras:** some types of layers (with some important parameters) :
  - **tf.keras.layers.Dense**(
    units, → number of neuron in the layer
    activation=None, → activation function
    use_bias=True, → use the bias
)
  - **tf.keras.layers.Conv2D**(note: there are also 1D and 3D versions
    filters, → number of neurons
    kernel_size, → kernel size (e.g. 3x3)
    strides=(1, 1), → step of kernel
    padding="valid", → padding if *"same"*
    activation=None, → activation function
    use_bias=True, → use the bias
)
    - Some activation functions: relu, sigmoid, softmax, tanh, ...

Artificial Intelligence for Robotics

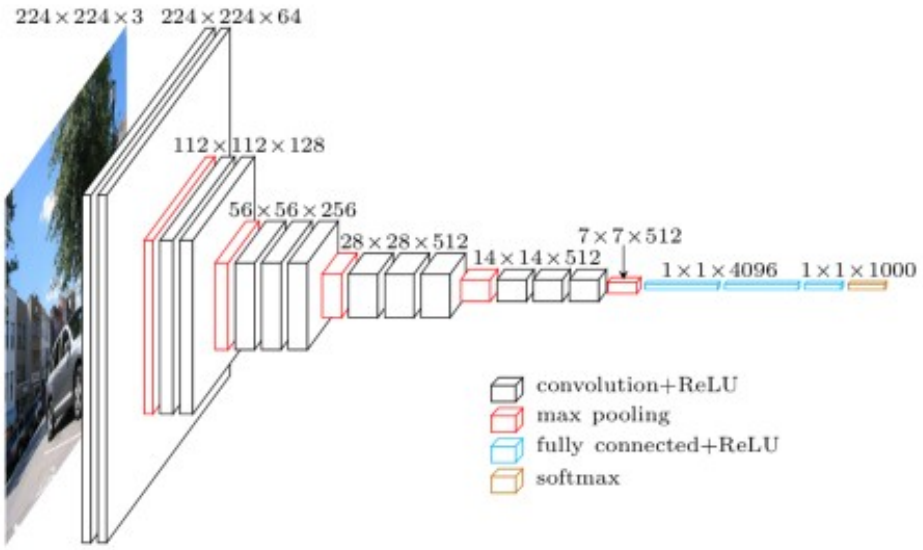
- **Keras:** some types of layers
  - **tf.keras.layers.MaxPooling2D(**
    pool_size=(2, 2), → reduction of the pooling (group size)
    strides=None, → step
    padding="valid"
)
  - **tf.keras.layers.UpSampling2D(**
    size=(2, 2), → increase of unpooling
    interpolation="nearest" → "nearest" or "bilinear"
)
  - **tf.keras.layers.Conv2DTranspose(**
    filters, → number of output images (depth)
    kernel_size, → kernel size
    strides=(1, 1), → adds 0 between pixels
    padding="valid", allows a ‘bed of nails’
    activation=None, with stride=(2,2)
    use_bias=True,
)



- Complete list of the 100+ types of layer on: https://keras.io/api/layers/

Artificial Intelligence for Robotics

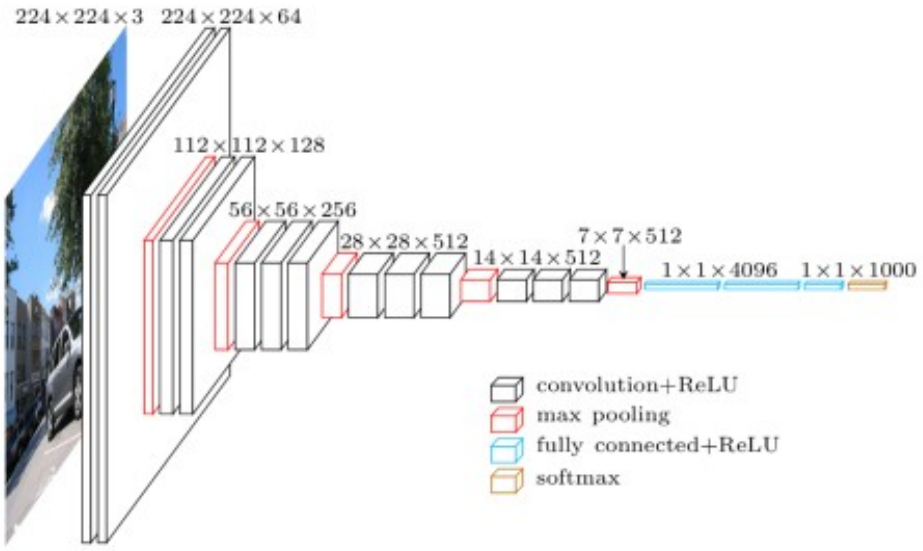
- **Keras**
  - Let’s implement VGG16 architecture:




```
model = Sequential( )

model.add(Input(shape=(224, 224, 3)))
model.add(Conv2D(filters=64, kernel_size=(3,3), padding="same", activation="relu"))
model.add(Conv2D(filters=64,kernel_size=(3,3), padding="same", activation="relu"))
model.add(MaxPool2D(pool_size=(2,2), strides=(2,2)))

model.add(Conv2D(filters=128, kernel_size=(3,3), padding="same", activation="relu"))
model.add(Conv2D(filters=128, kernel_size=(3,3), padding="same", activation="relu"))
model.add(MaxPool2D(pool_size=(2,2),strides=(2,2)))

model.add(Conv2D(filters=256, kernel_size=(3,3), padding="same", activation="relu"))      X 3
model.add(MaxPool2D(pool_size=(2,2),strides=(2,2)))

model.add(Conv2D(filters=512, kernel_size=(3,3), padding="same", activation="relu"))      X 3
model.add(MaxPool2D(pool_size=(2,2),strides=(2,2)))
```

Artificial Intelligence for Robotics

- **Keras**
  - Let’s implement VGG16 architecture:



...

```
model.add(Conv2D(filters=512, kernel_size=(3,3), padding="same", activation="relu"))
model.add(Conv2D(filters=512, kernel_size=(3,3), padding="same", activation="relu"))
model.add(Conv2D(filters=512, kernel_size=(3,3), padding="same", activation="relu"))
model.add(MaxPool2D(pool_size=(2,2),strides=(2,2)))

model.add(Flatten( ))

model.add(Dense(units=4096,activation="relu"))
model.add(Dense(units=4096,activation="relu"))

model.add(Dense(units=2, activation="softmax"))
```

Artificial Intelligence for Robotics

- **Keras**
  - A sequential model can also be implemented as a vector of layers:

```
model = Sequential( [
        Conv2D(input_shape=(224,224,3), filters=64, kernel_size=(3,3), padding="same", activation="relu"),
        Conv2D(filters=64,kernel_size=(3,3), padding="same", activation="relu"),
        MaxPool2D(pool_size=(2,2), strides=(2,2)), …
] )
```

  - The functional model
    - Layers are created independently, then outputs and inputs are connected

```
input_layer = Input(shape=(28,28,1))
conv1 = Conv2D(32, kernel_size=(3, 3), activation='relu', input_shape=(32,32,1))(input_layer)
conv2 = Conv2D(64, (3, 3), activation='relu', input_shape=(32,32,1))(input_layer)
merge = concatenate([conv1, conv2])
flatten = Flatten( )(merge)
output_layer = Dense(num_classes, activation='softmax')(flatten)
model = Model(inputs=input_layer, outputs=output_layer)
```

    - Enable non-sequential models (fusion, split...)

Artificial Intelligence for Robotics

- **Keras: training parameters**
  - We select the error function (loss) :

```
loss=keras.losses.CategoricalCrossentropy()
```

    Also:
    - *MeanSquaredError :* $E = (\hat{y} - y)^2$
    - … and some other dozens on https://keras.io/api/losses
  - In case of label vectors ([1, 5, 3, 0, 6, …]), they must be converted into one-hot vector for comparison with output of last layer
    - We use function *to_categorical* :

```
output_train=keras.utils.to_categorical(output_train, num_classes=10)
```

    → [[0,1,0,0,0,0,0,0,0,0],[0,0,0,0,1,0,0,0,0,0],[0,0,1,0,0,0,0,0,0,0],…]

    - Note: the loss function *SparseCategoricalCrossentropy* is a function CategoricalCrossentropy that directly use labels

Artificial Intelligence for Robotics

- **Keras: training parameters**

  - We select the optimization function:

```
optim = keras.optimizers.SGD(learning_rate=0.01)
```

    and some more efficient: Adam, Adadelta, Nadam, Ftrl...
    (see https://keras.io/api/optimizers/)

  - Then we compile the network:

```
model.compile(loss=loss, optimizer=optim, metrics=["accuracy"])
```

Artificial Intelligence for Robotics

- **Keras: Training the network**
  - We use the function *fit* :

```
model.fit(img_train, output_train, batch_size=64, epochs=10, shuffle=True, verbose=2)
```

  - img_train and output_train: dataset and expected output
  - batch_size: number of test example before updating weights
  - Epochs : number of epochs
  - Shuffle : the dataset can be shuffled at each epock
  - Verbose : level of data displayed during training

```
Epoch 6/10
938/938 - 1s - loss: 0.0361 - accuracy: 0.9889
Epoch 7/10
938/938 - 1s - loss: 0.0354 - accuracy: 0.9892
Epoch 8/10
938/938 - 1s - loss: 0.0348 - accuracy: 0.9894
Epoch 9/10
938/938 - 1s - loss: 0.0343 - accuracy: 0.9896
Epoch 10/10
938/938 - 1s - loss: 0.0338 - accuracy: 0.9898
```

Artificial Intelligence for Robotics

- **Keras : evaluation and exploitation**
  - The network can be evaluated on an evaluation dataset with *evaluate*

```
model.evaluate(img_test,output_test, batch_size=16, verbose=2)
```

```
10/10 - 0s - loss: 0.1384 - accuracy: 0.9654
[0.1383775770664215, 0.9653999805450044]
```

  - We can then use the network to make predictions:

```
model.predict(img_test[0:1,:,:])
```

```
array([[1.0455251e-03, 1.4841557e-04, 5.5478072e-01, 9.7418678e-01,
        7.2960603e-07, 2.4808049e-03, 8.8775799e-11, 9.9999410e-01,
        1.3713300e-02, 2.5117993e-02]], dtype=float32)
```

Result can also be obtained as a tensor: `model(img_test[0:1,:,:])`

Artificial Intelligence for Robotics

- **Conclusion :**

  - Deep learning is a recent domain with significant margin of progression

  - A neural network however remains a classifier, unable to understand or interpret its results, nor presented data. It is also limited by the training dataset.
  - An AI must, to properly develop itself, explore and experiment its environment by itself.

    → On the way to reinforcement learning?